

Attention

Yu LI (李煜)

Thursday, 29 October 2026

liyu95.com

liyu@cse.cuhk.edu.hk

Department of Computer Science and Engineering (CSE)

The Chinese University of Hong Kong (CUHK)

Comments and Questions



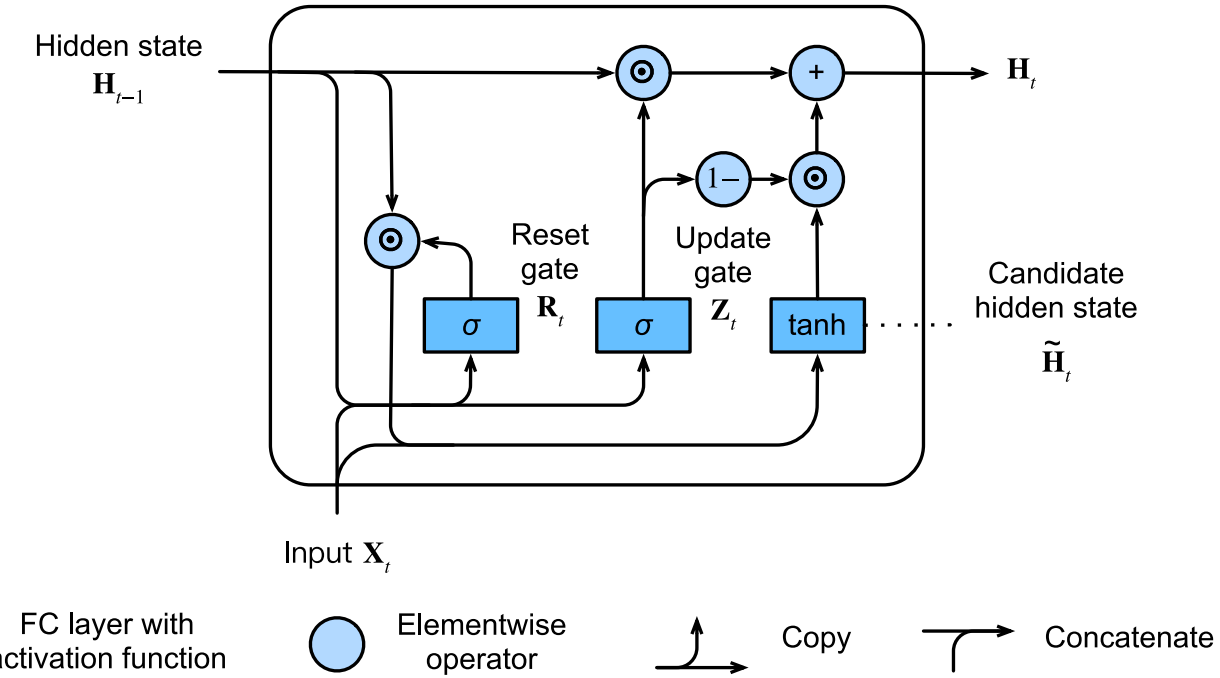
- NA
- Can we add a form receipt for the late day submission? I personally feel afraid without seeing the confirmation that I have submitted it successfully and sometimes I even forgot have I done so last time I emailed TA to ask if my late day application went in successfully but I don't think that is a good use of TA's time.
  - **The TA will do that and send out an email.**
- good contents
- Good
- Interesting and useful
- Already good

More prior knowledge



- Text can be long
  - A *recurrent hidden cell*: learn the pattern
- Early observation may be very important
  - Truncation may miss such information. It may also decay in the long sequence
  - A *memory cell*: remember important information
- Some information is meaningless
  - HTML tags
  - *Skip* such meaningless tokens
- Logic break
  - A new chapter
  - *Reset* our hidden state

Gated recurrent units (GRUs)



Memorize, Skip, Reset

How much we use $\boldsymbol{X}_t$ and $\boldsymbol{H}_{t-1}$ to get $\boldsymbol{H}_t$

$$\boldsymbol{H}_t = \phi(\boldsymbol{X}_t\boldsymbol{W}_{xh} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$

We need some gates to turn the information on or off

Reset gates help capture short-term dependencies in sequences

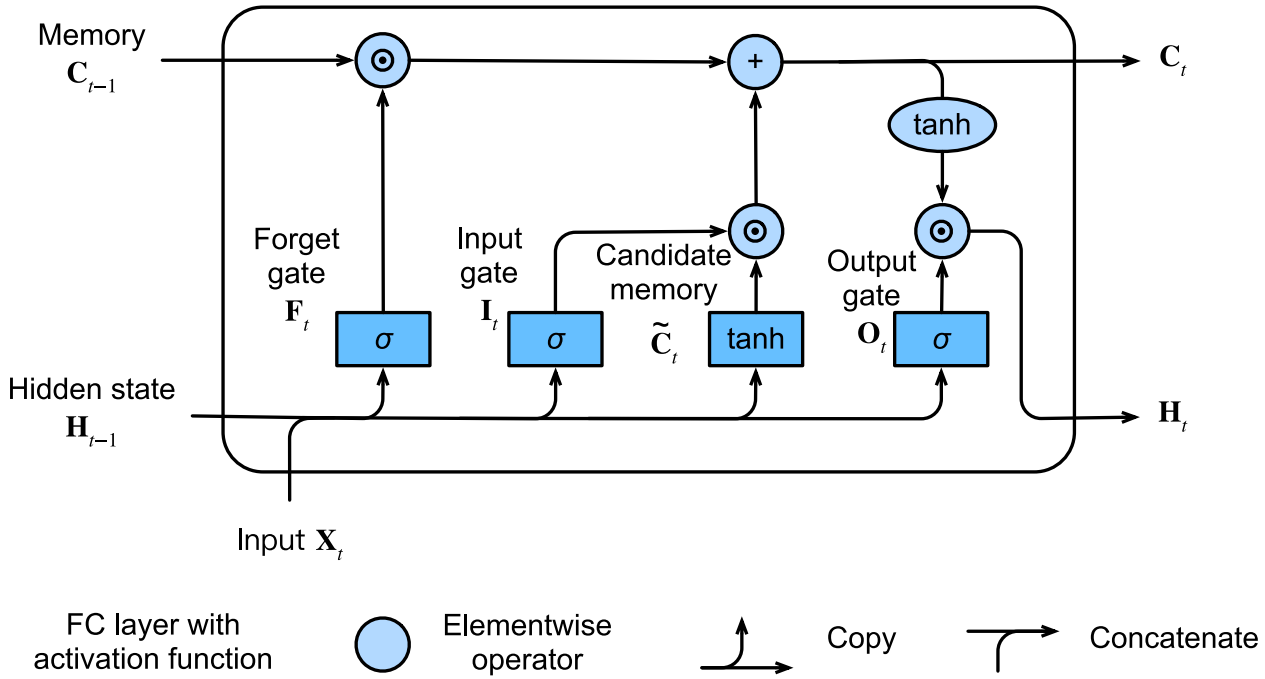
Update gates help capture long-term dependencies in sequences

$$\boldsymbol{H}_t = (1 - \boldsymbol{Z}_t)\odot\tilde{\boldsymbol{H}}_t + \boldsymbol{Z}_t\odot\boldsymbol{H}_{t-1}$$
$$\tilde{\boldsymbol{H}}_t = tanh(\boldsymbol{X}_t\boldsymbol{W}_{xh} + (\boldsymbol{R}_t\odot\boldsymbol{H}_{t-1})\boldsymbol{W}_{hh} + \boldsymbol{b}_h)$$
$$\boldsymbol{R}_t = \delta(\boldsymbol{X}_t\boldsymbol{W}_{xr} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hr} + \boldsymbol{b}_r)$$
$$\boldsymbol{Z}_t = \delta(\boldsymbol{X}_t\boldsymbol{W}_{xz} + \boldsymbol{H}_{t-1}\boldsymbol{W}_{hz} + \boldsymbol{b}_z)$$

Analysis

$\mathbf{R}_t$, $\mathbf{Z}_t$: 0, 1

Long short-term memory (LSTM)



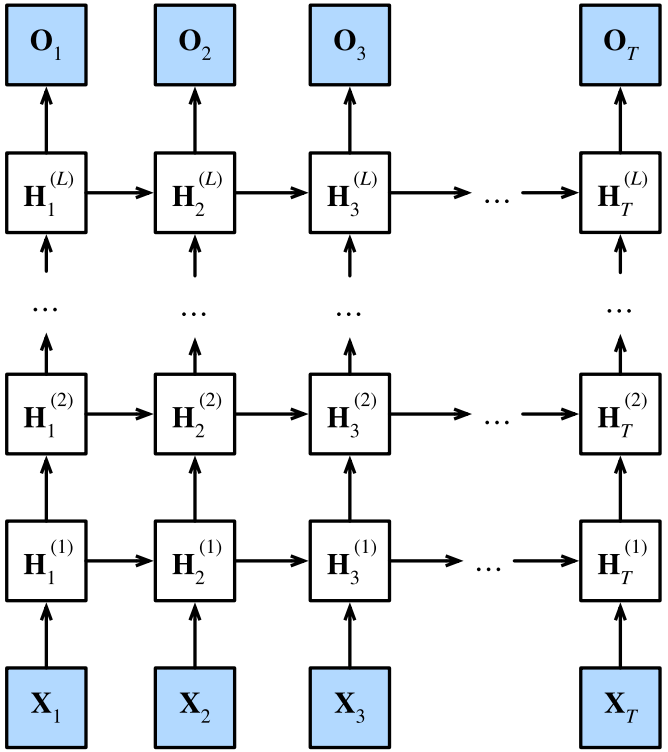
$$\boldsymbol{H}_t = \boldsymbol{O}_t \odot tanh(\boldsymbol{C}_t)$$
$$\boldsymbol{C}_t = \boldsymbol{F}_t \odot \boldsymbol{C}_{t-1} + \boldsymbol{I}_t \odot \tilde{\boldsymbol{C}}_t$$
$$\tilde{\boldsymbol{C}}_t = tanh(\boldsymbol{X}_t \boldsymbol{W}_{xc} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hc} + \boldsymbol{b}_c)$$
$$\boldsymbol{I}_t = \delta(\boldsymbol{X}_t \boldsymbol{W}_{xi} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hi} + \boldsymbol{b}_i)$$
$$\boldsymbol{F}_t = \delta\left(\boldsymbol{X}_t \boldsymbol{W}_{xf} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{hf} + \boldsymbol{b}_f\right)$$
$$\boldsymbol{O}_t = \delta(\boldsymbol{X}_t \boldsymbol{W}_{xo} + \boldsymbol{H}_{t-1} \boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

Deep RNN/LSTM/GRU

Hopefully,

High-level trend
(bear or bull market)

Shorter-term
temporal dynamics
(data of the day)

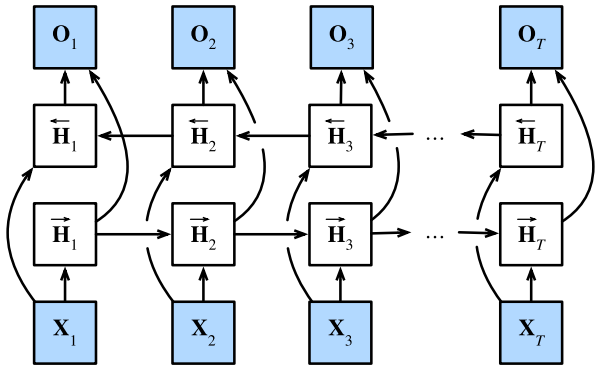


Increase the model capacity

$$\boldsymbol{H}_t^{(l)} = \phi_l\left(\boldsymbol{H}_t^{(l-1)}\boldsymbol{W}_{xh}^{(l)} + \boldsymbol{H}_{t-1}^{(l)}\boldsymbol{W}_{hh}^{(l)} + \boldsymbol{b}_h^{(l)}\right)$$

$$\boldsymbol{O}_t = \delta(\boldsymbol{H}_t^{(L)}\boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

Bidirectional model





- Limited applications
  - Can't be used to do forward prediction
  - Training with information of both direction, testing only one direction. Poor performance
  - Filling in missing words
  - Annotating tokens: name entity recognition
  - Sequence encoding: machine translation

$$\overrightarrow{\boldsymbol{H}}_t = \phi\left(\boldsymbol{X}_t \boldsymbol{W}_{xh}^{(f)} + \overrightarrow{\boldsymbol{H}}_{t-1} \boldsymbol{W}_{hh}^{(f)} + \boldsymbol{b}_h^{(f)}\right)$$

$$\overleftarrow{\boldsymbol{H}}_t = \phi\left(\boldsymbol{X}_t \boldsymbol{W}_{xh}^{(b)} + \overleftarrow{\boldsymbol{H}}_{t+1} \boldsymbol{W}_{hh}^{(b)} + \boldsymbol{b}_h^{(b)}\right)$$

$$\boldsymbol{O}_t = \delta(cat(\overrightarrow{\boldsymbol{H}}_t, \overleftarrow{\boldsymbol{H}}_t)\boldsymbol{W}_{ho} + \boldsymbol{b}_o)$$

- Slow
  - Very, very slow
  - Forward propagation requires both forward and backward recursions

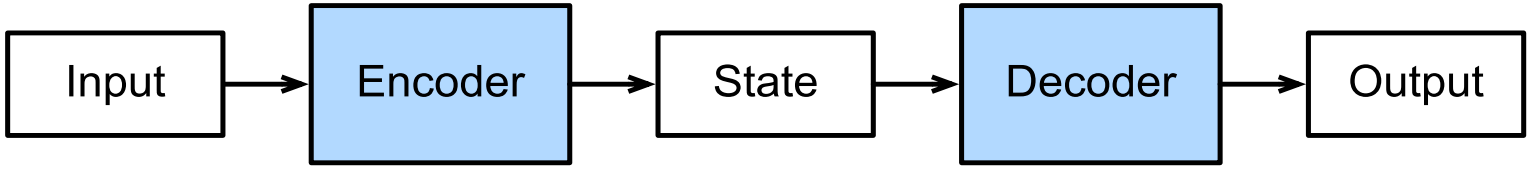
Machine translation



- Sequences of different lengths
  - Not direct mapping

- Encoder: transform the input into a state with a fixed shape

- Decoder: map the encoded state to a variable-length sequence



Sequence to sequence model

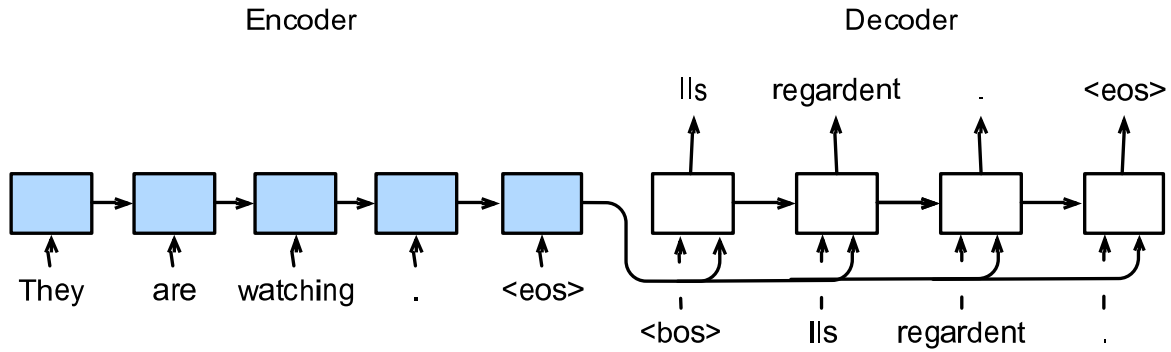


Encoder

$$\boldsymbol{h}_t = f(\boldsymbol{x}_t, \boldsymbol{h}_{t-1})$$
$$\boldsymbol{c} = q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T)$$

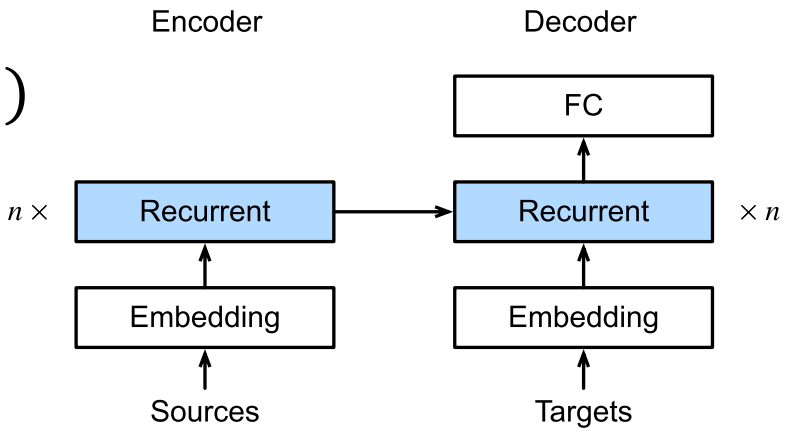
For example,

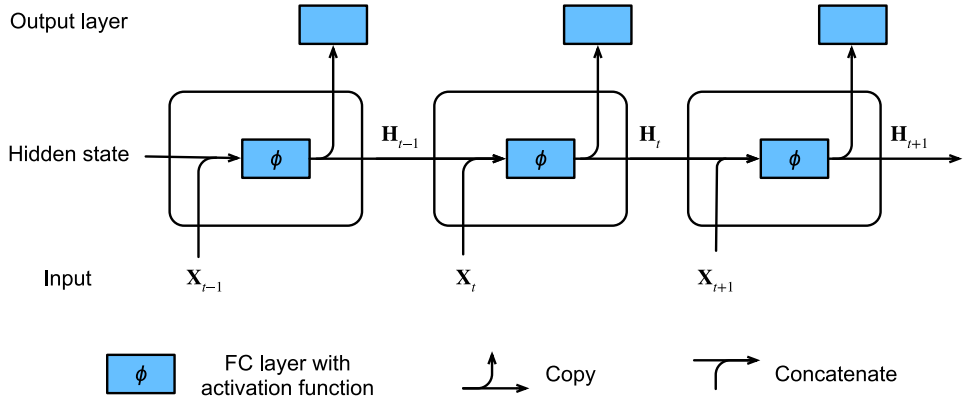
$$\boldsymbol{c} = q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T) = \boldsymbol{h}_T$$



Decoder

$$\boldsymbol{s}_{t'} = g(y_{t'}, \boldsymbol{c}, \boldsymbol{s}_{t'-1})$$

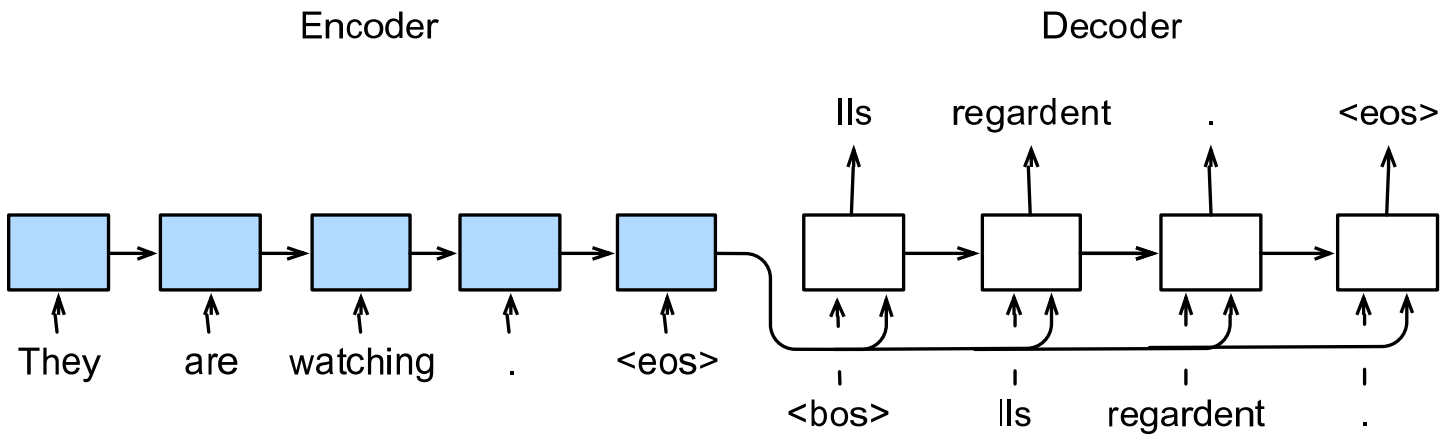


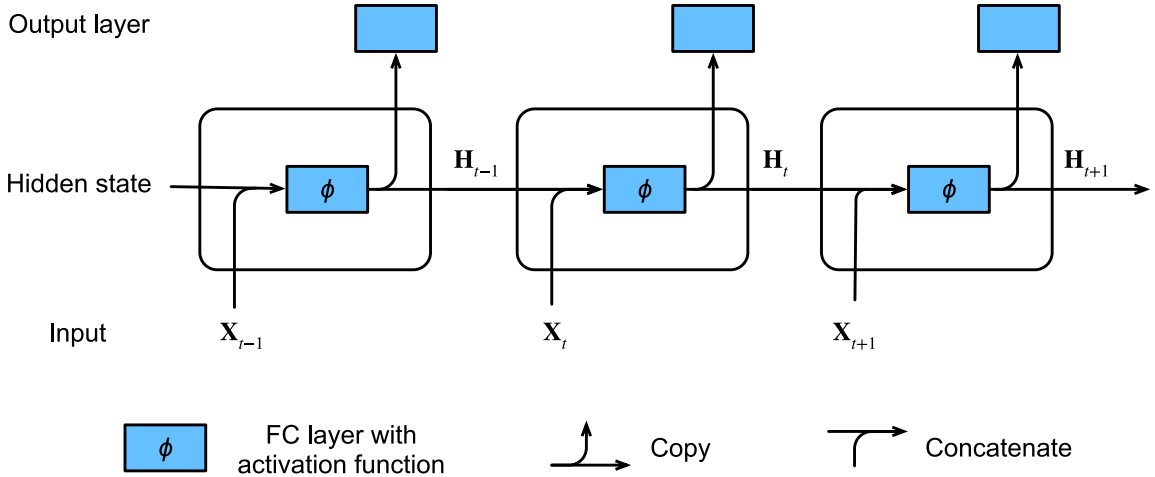


Today's agenda

- Attention
  - Attention mechanism
  - Self-attention
  - Multi-head attention
  - Position embedding
  - Transformer

Machine translation





Hidden state is very important

We only have one hidden state to aggregate the information

The hidden state flows across the sentence

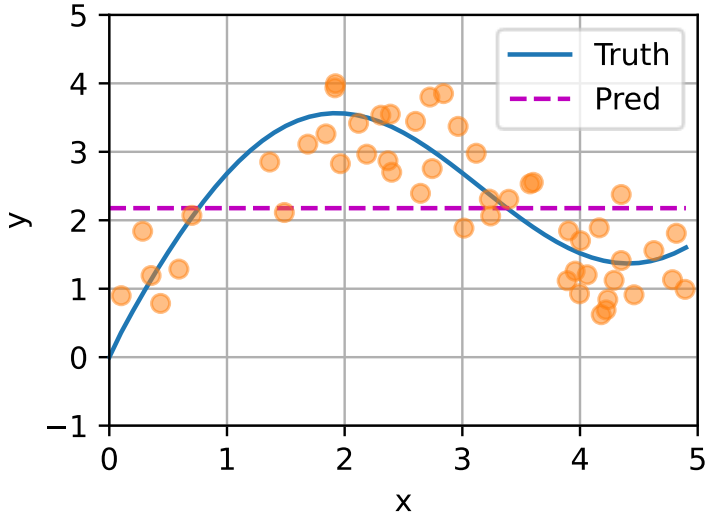
Can't ensure the representation for each word is proper

Any idea of encoding each word?

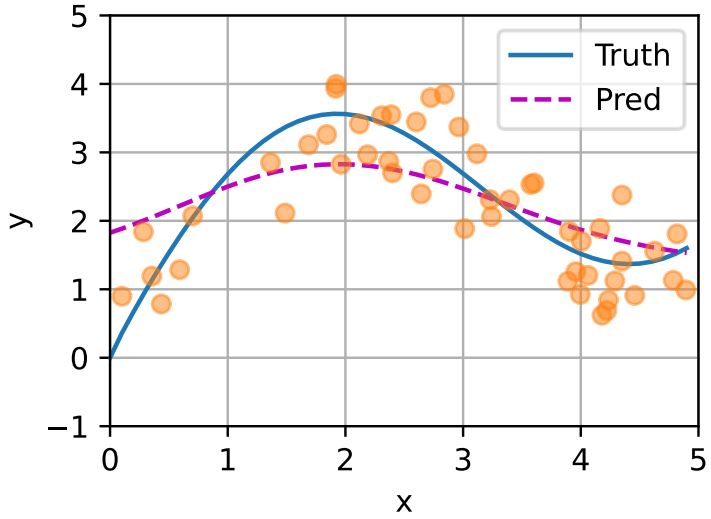
Representation using neighbor information



- **Average**
  - $f(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$

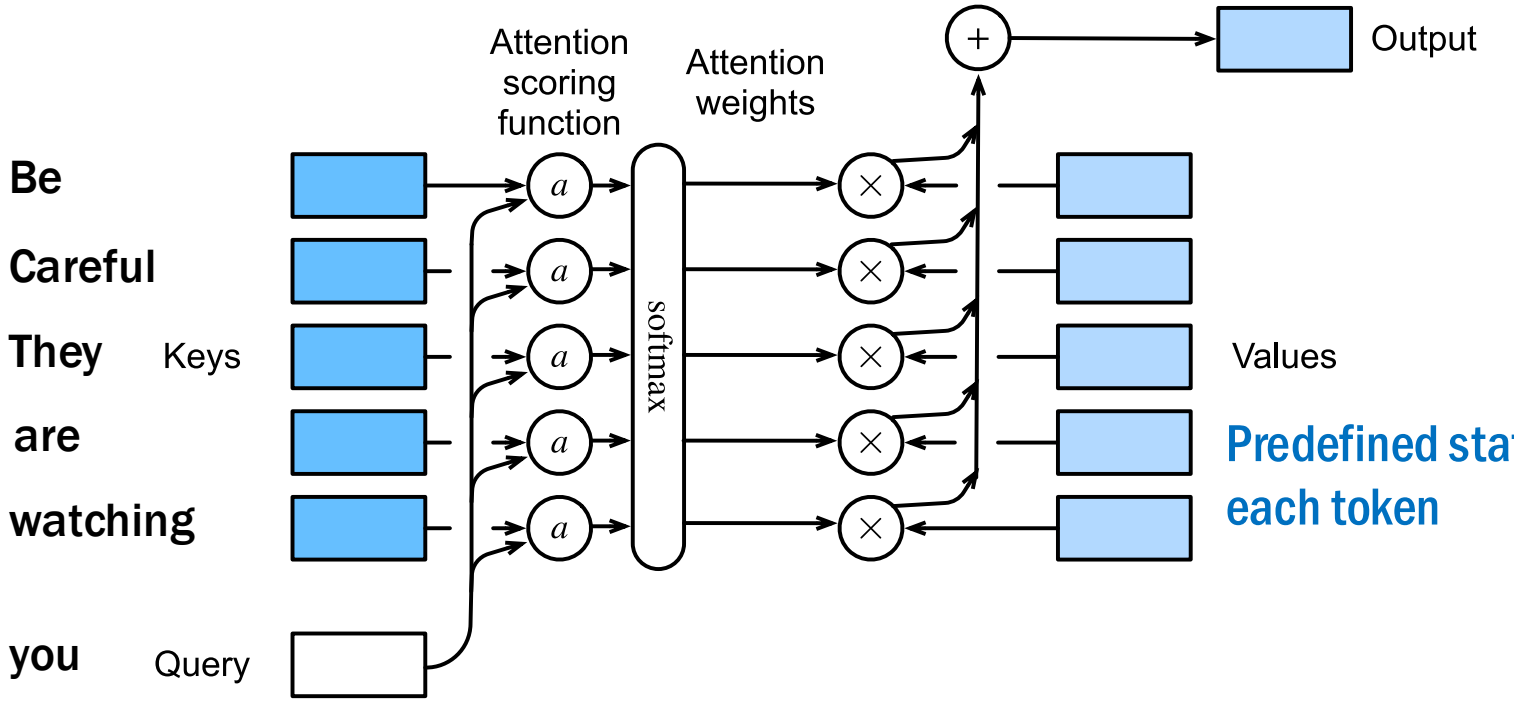


- **Weighted average**
  - $f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$
  - $\alpha(x, x_i)$: related to similarity or distance
  - Even more general: Attention!



Attention mechanism: Machine translation

Hidden state of each word: considering interactions with all the other words directly



Hidden state of Query

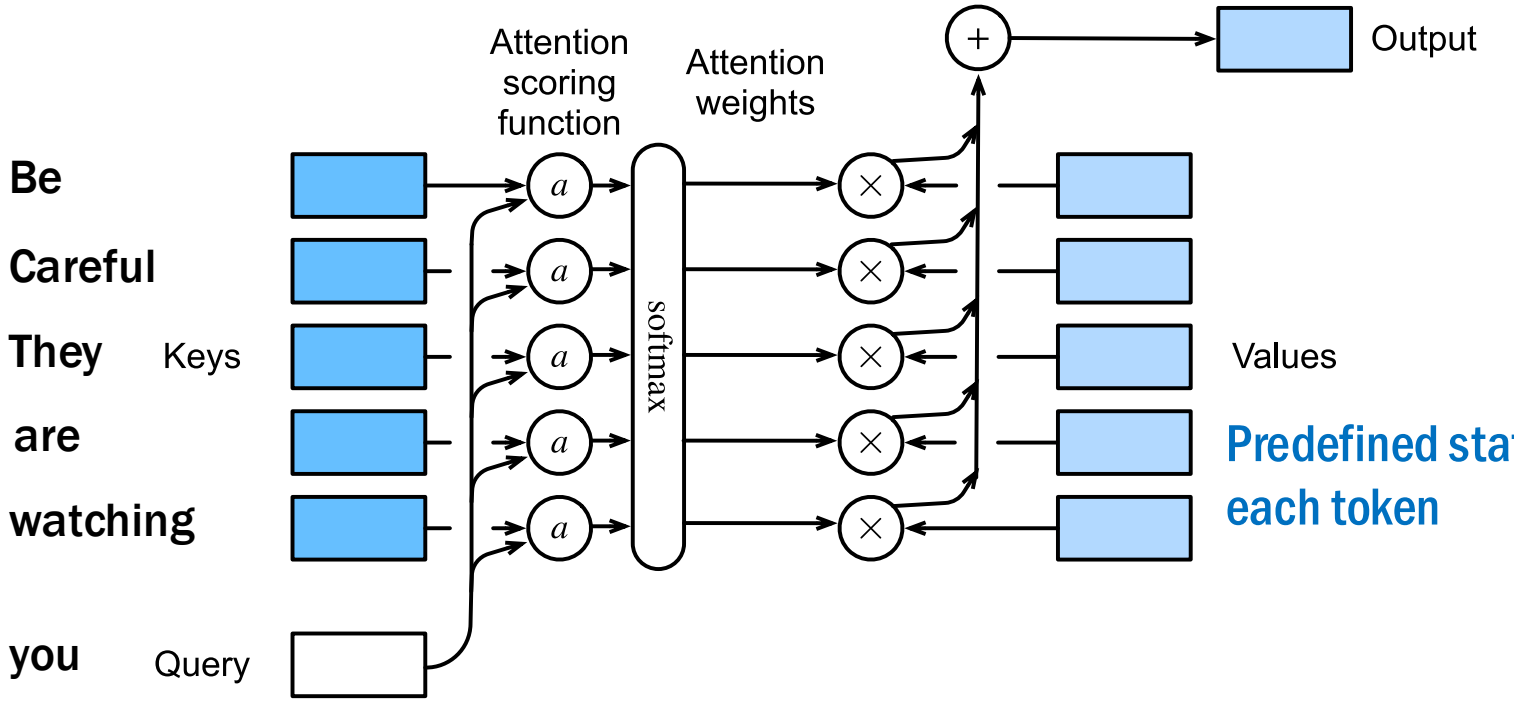
Based on the hidden state, we can do the downstream task

Predefined state for each token

$$Query\ hidden\ state\ =\ f(x)\ =\ \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

Attention mechanism: Machine translation

Hidden state of each word: considering interactions with all the other words directly



Hidden state of Query

Based on the hidden state, we can do the downstream task

Predefined state for each token

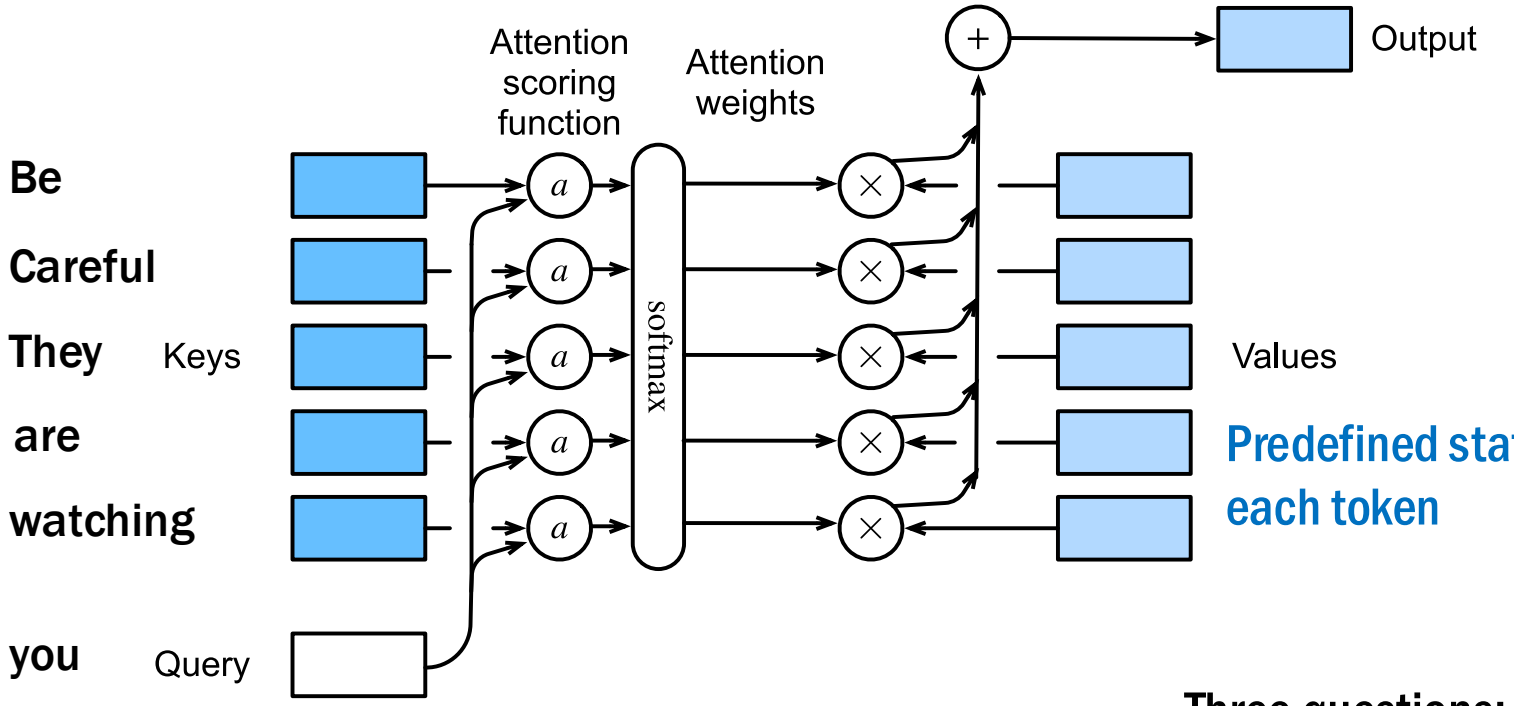
$$Query\ hidden\ state\ =\ f(x)\ =\ \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

What is the difference between output hidden state and predefined state?

Context and neighbor information

Attention mechanism: Machine translation

Hidden state of each word: considering interactions with all the other words directly



Hidden state of Query

Based on the hidden state, we can do the downstream task

Predefined state for each token

$$Query\ hidden\ state\ =\ f(x)\ =\ \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

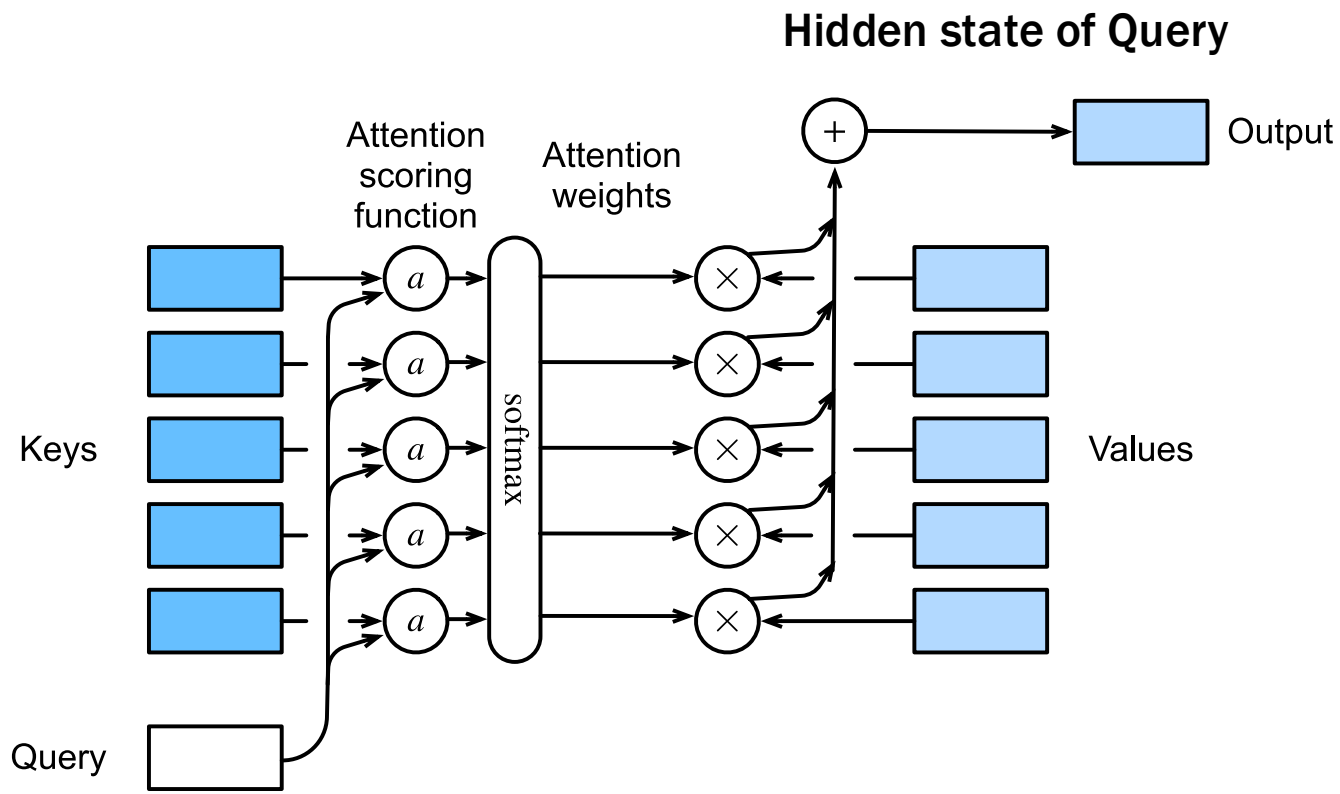
Three questions:

1. How to encode character?
2. How to define the predefined state?
3. How to define the attention weight function?

Attention mechanism: Machine translation



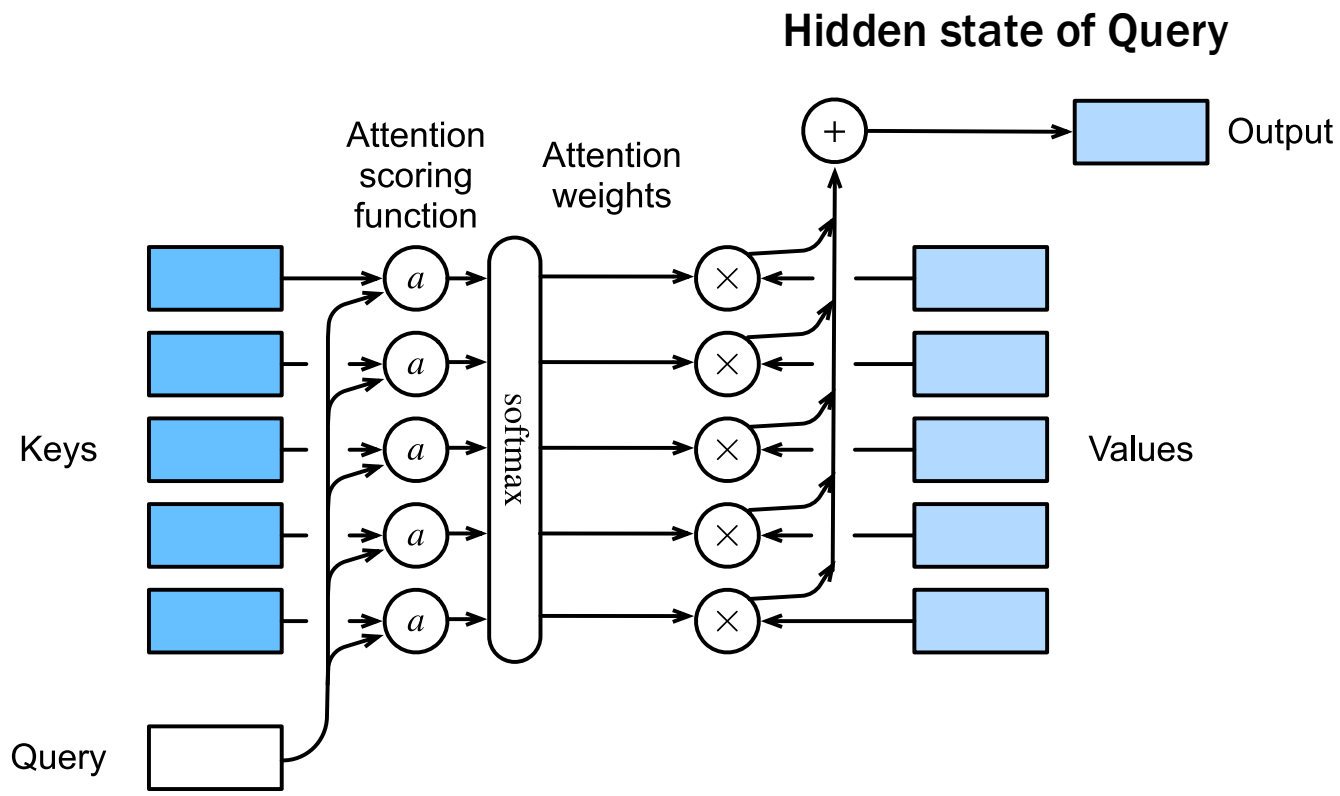
- Character encoding (keys)
  - One-hot encoding
  - Or FCNet (one-hot)
- Values
  - FCNet (one-hot)
- Attention weights (dot-product)
  - $\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))}$
  - $a(\boldsymbol{q}, \boldsymbol{k}) = \frac{\boldsymbol{q}^{\mathrm{T}} \boldsymbol{k}}{\sqrt{\mathrm{d}}}$



Attention mechanism: Machine translation



- Character encoding (keys)
  - One-hot encoding
  - Or FCNet (one-hot)
- Values
  - FCNet (one-hot)
- Attention weights (dot-product)
  - $\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_j))}$
  - $a(\boldsymbol{q}, \boldsymbol{k}) = \frac{\boldsymbol{q}^{\mathrm{T}} \boldsymbol{k}}{\sqrt{\mathrm{d}}}$

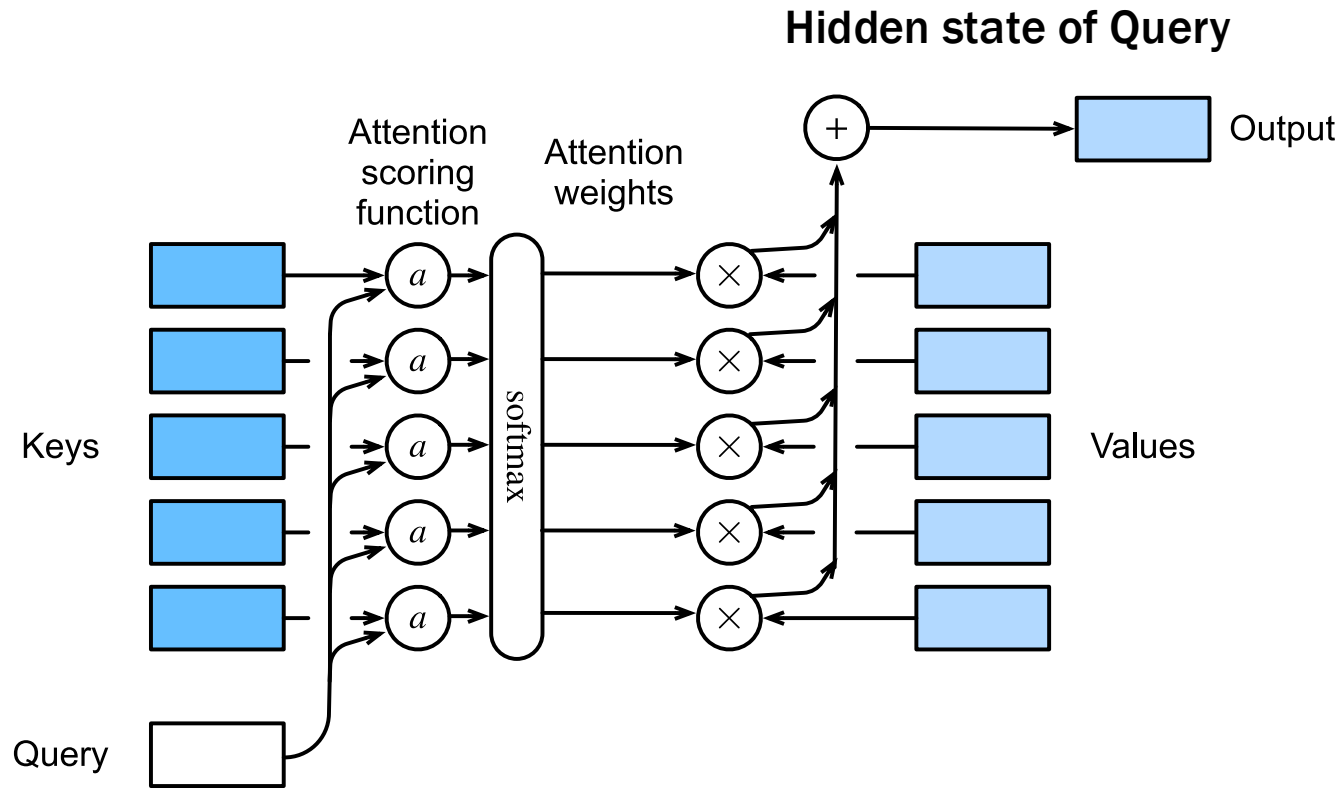


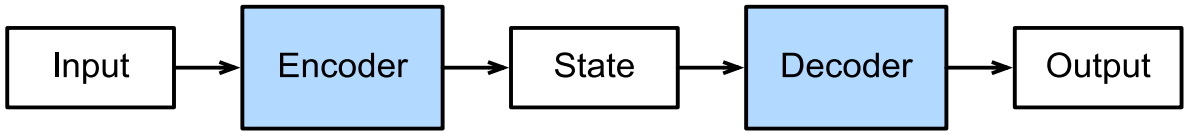
What are the learnable parameters in attention?

Attention mechanism: Machine translation



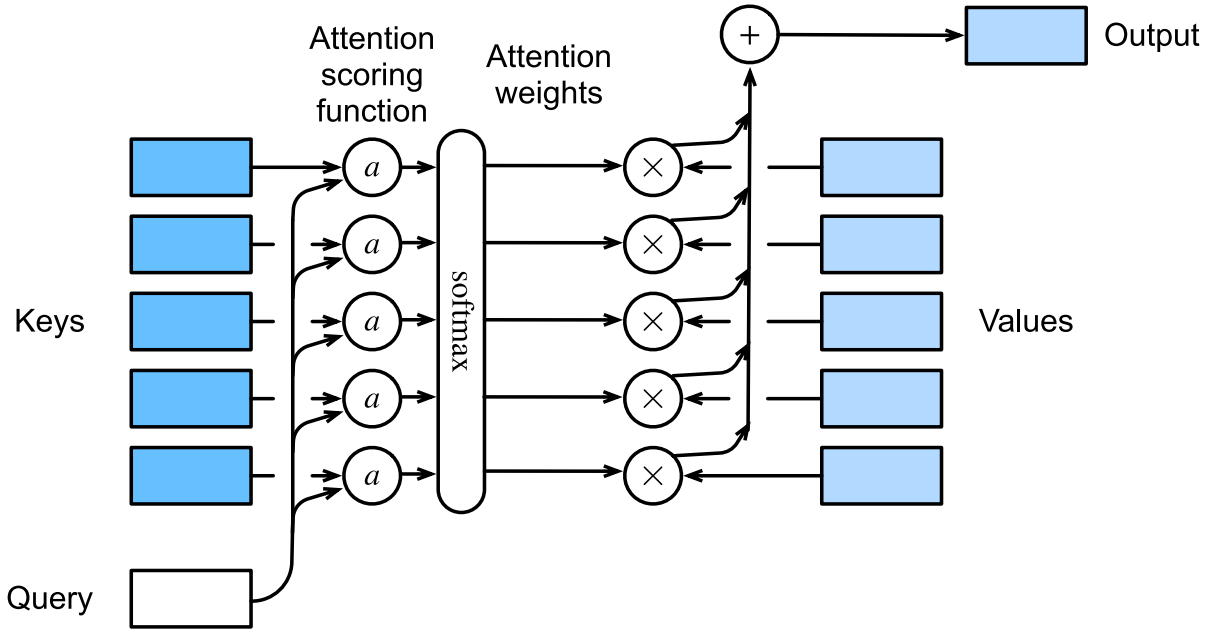
- Loss function
  - Cross-entropy for each word prediction
- Training
  - The common optimizers
  - Back-propagation





How to get the hidden state of all the tokens in the sentence?

Self-attention



For each input $\boldsymbol{x}_i$ in the sentence: $\boldsymbol{y}_i = \sum_{j=1}^{m} \alpha(\boldsymbol{x}_i, \boldsymbol{x}_j)\boldsymbol{v}_j$

$\boldsymbol{y}_i$: outputted hidden state for downstream classification or other tasks

Including the attention against the $\boldsymbol{x}_i$ itself

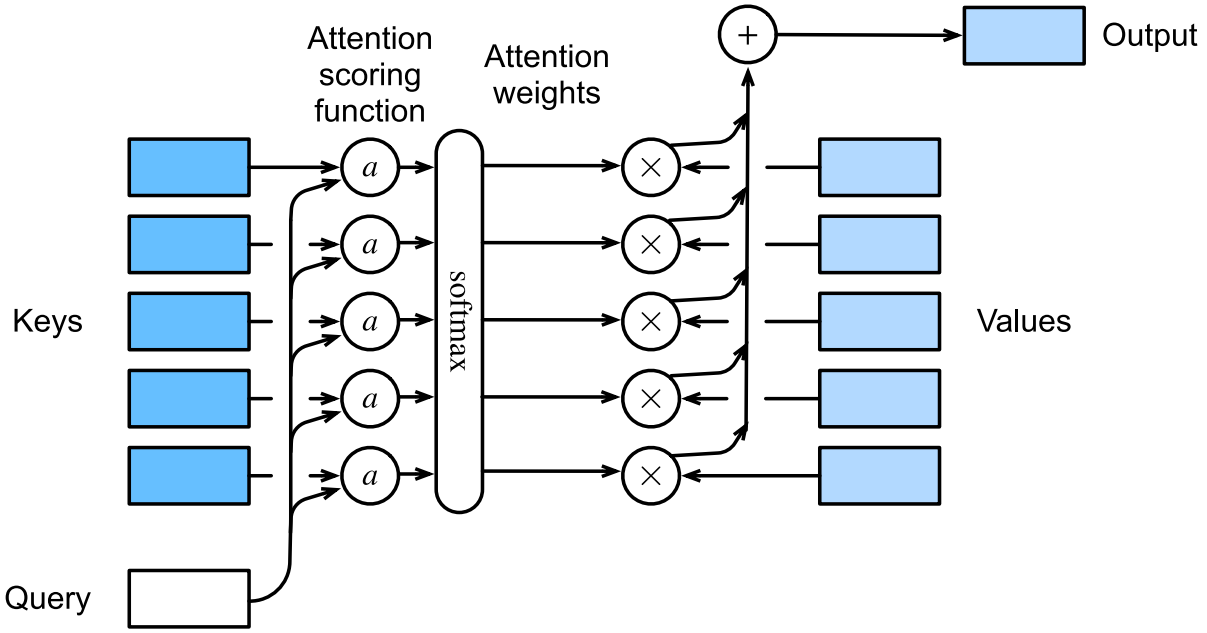
Can be computed in parallel

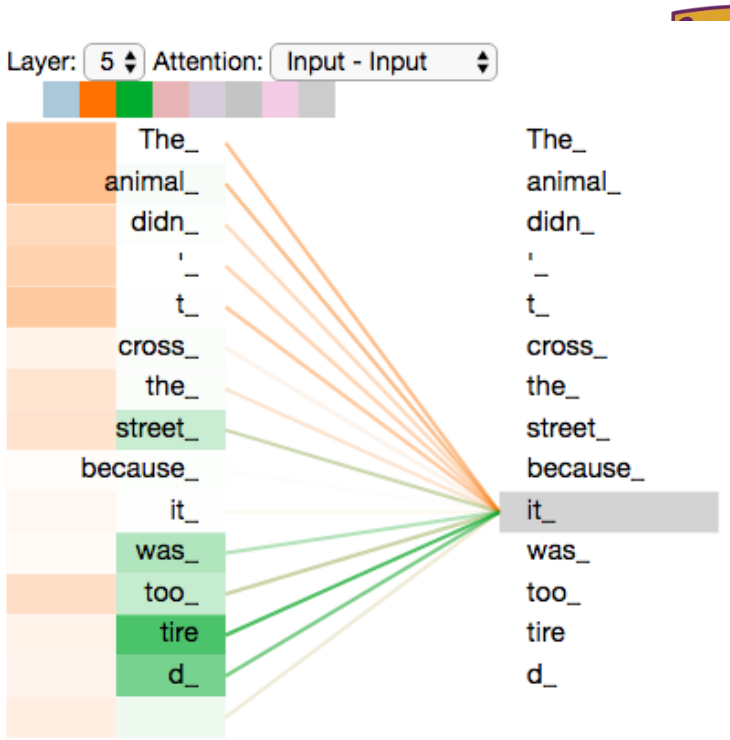
Get the encoding of each token by considering the context

Attention against the sentence itself

Encoding and embedding are the most important things for NLP

Self-attention





For each input x_i in the sentence: $y_i = \sum_{j=1}^{m} \alpha(x_i, x_j) v_j$

y_i: outputted hidden state for downstream classification or other tasks

Including the attention against the x_i itself

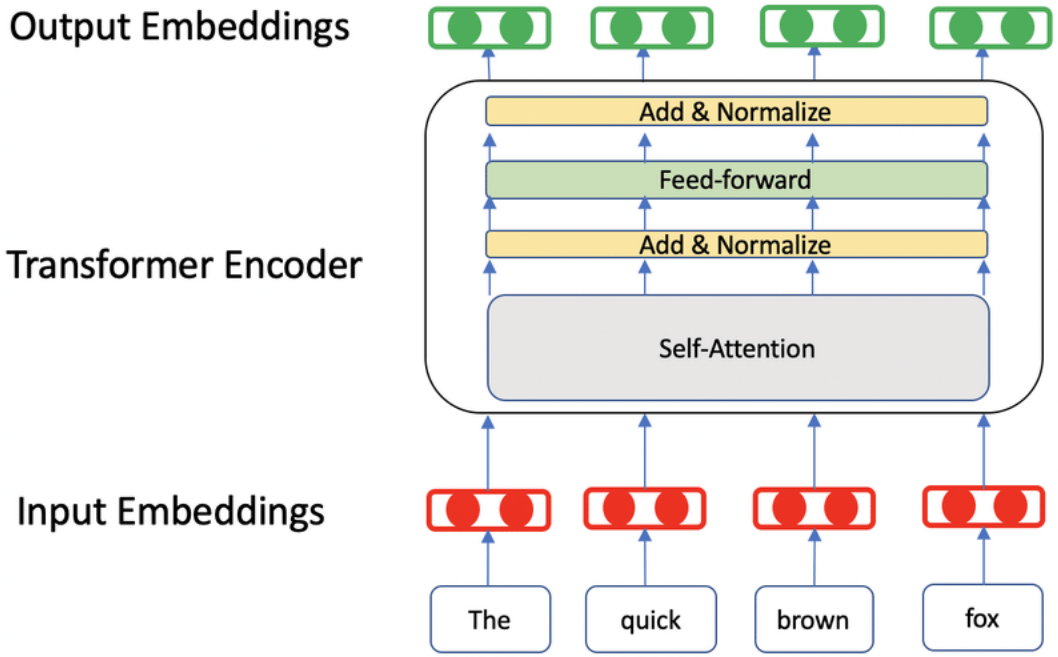
Can be computed in parallel

Get the encoding of each token by considering the context

Attention against the sentence itself

Encoding and embedding are the most important things for NLP

Self-attention



For each input $\boldsymbol{x}_i$ in the sentence: $\boldsymbol{y}_i = \sum_{j=1}^{m} \alpha(\boldsymbol{x}_i, \boldsymbol{x}_j)\boldsymbol{v}_j$

$\boldsymbol{y}_i$: outputted hidden state for downstream classification or other tasks

Including the attention against the $\boldsymbol{x}_i$ itself

Can be computed in parallel

Long-term dependency?



Time series

Sequences

Text

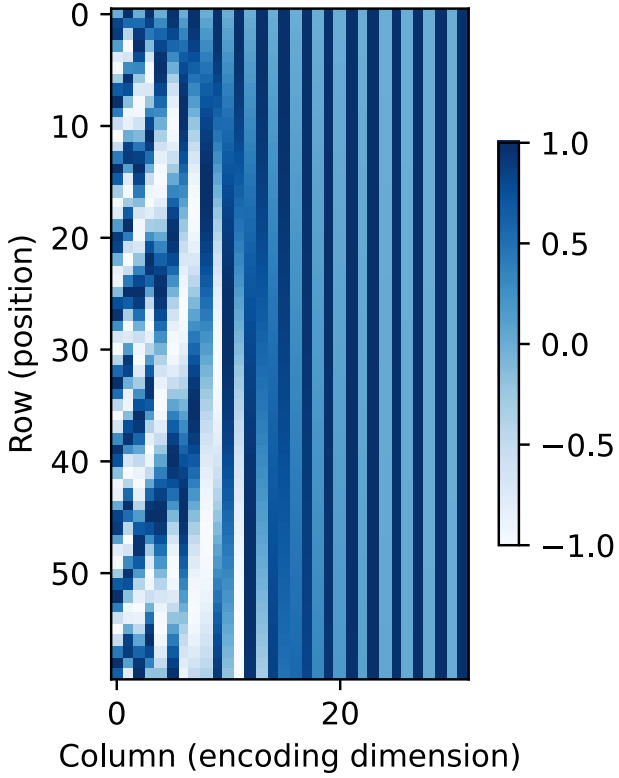
...

Order information

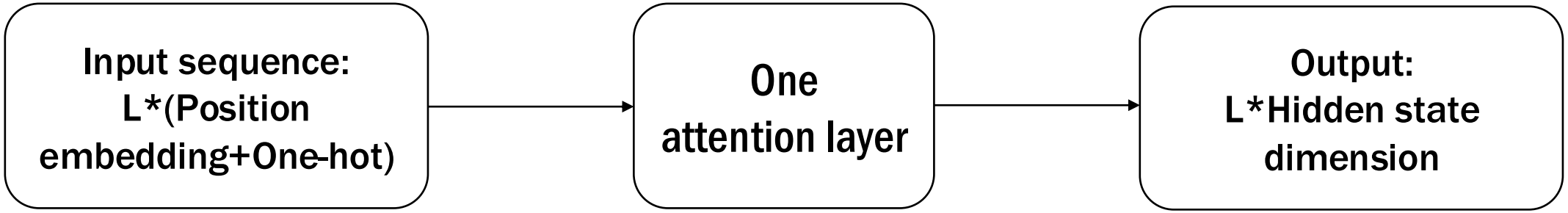
Position encoding



- $\boldsymbol{X} \in R^{n*d}$: d-dimensional embeddings for n tokens of a sequence
- $\mathbf{P} \in R^{n*d}$
  - The value of the ith row and the $(2j)^{th}$ or the $(2j+1)^{th}$ column
  - $p_{i,2j} = \sin(\frac{i}{10000^{\frac{2j}{d}}})$
  - $p_{i,2j+1} = \cos(\frac{i}{10000^{\frac{2j}{d}}})$
- Weird function but people use it
- Absolute & relative positional information
- Input dimension = Raw encoding dimension + PE dimension



Input and output of attention layer



Increase complexity?

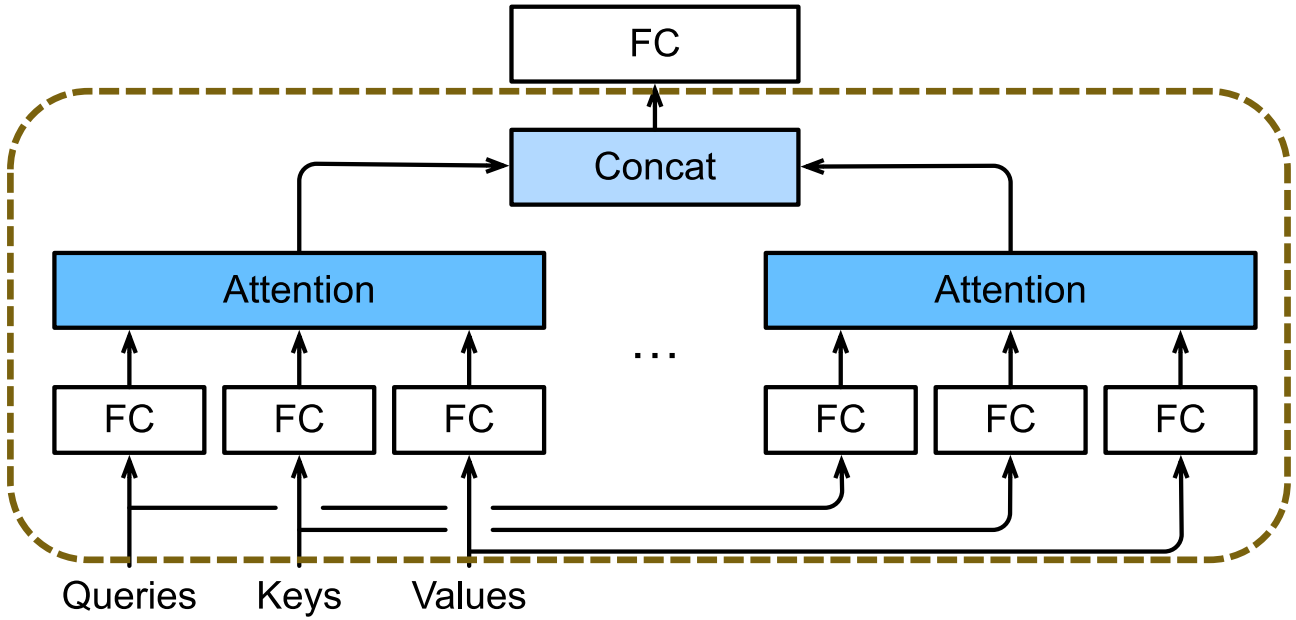
Multi-head attention and stack more attention layers

Multi-head attention

Why do we need FCNet here?

Where do we apply the FC? Token or sentence?

How is the FC parameter shared?

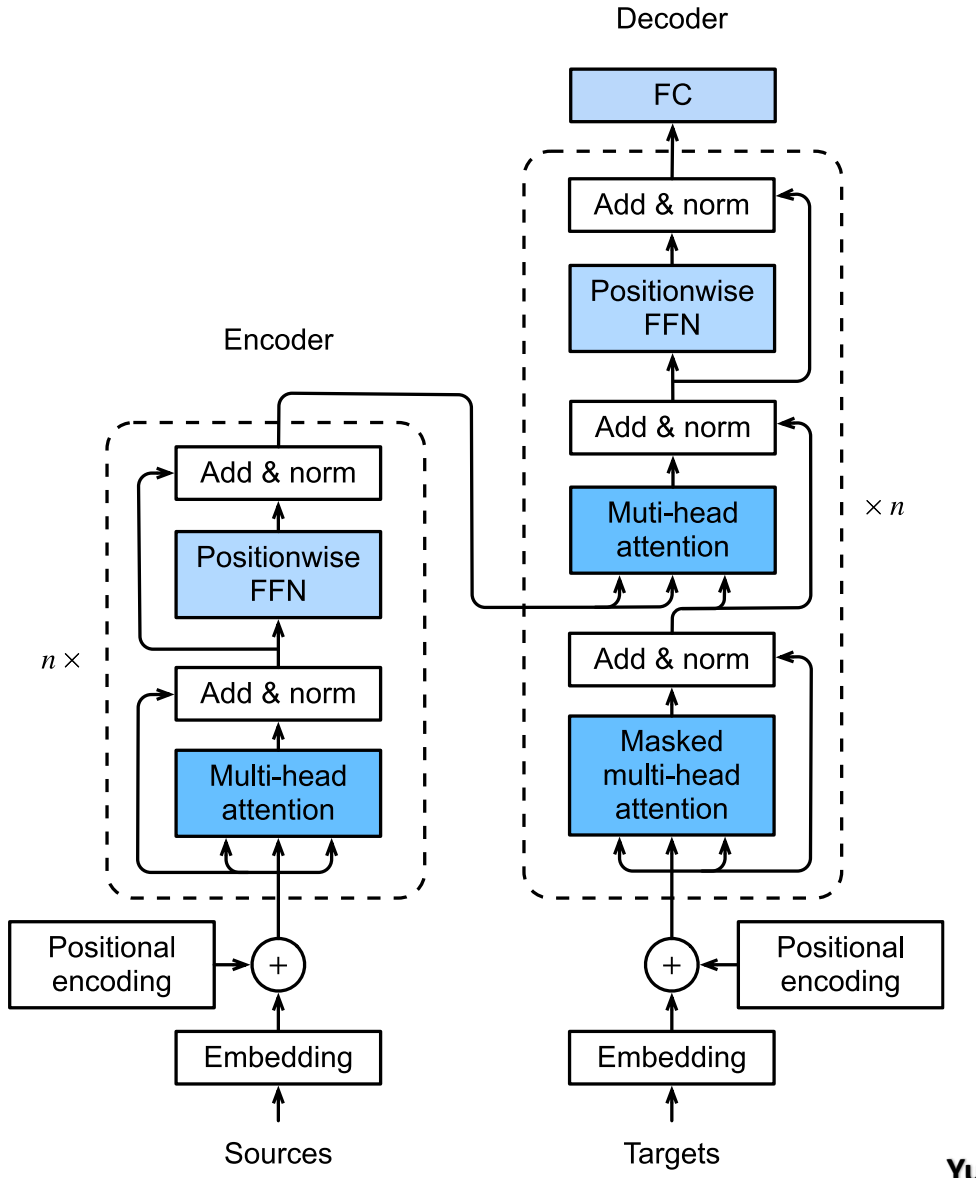


L*Hidden state dimension

L*(Hidden state dimension*n_head)

L*Hidden state dimension

Transformer for machine translation



Within each block:

Multi-head attention to learn the hidden state

FCNet to control dimension (for easy implementation)

Multiple blocks: increase complexity

Masked attention: do not consider padding position

You can understand it fully only after you have read their code and run the program

To make you awake

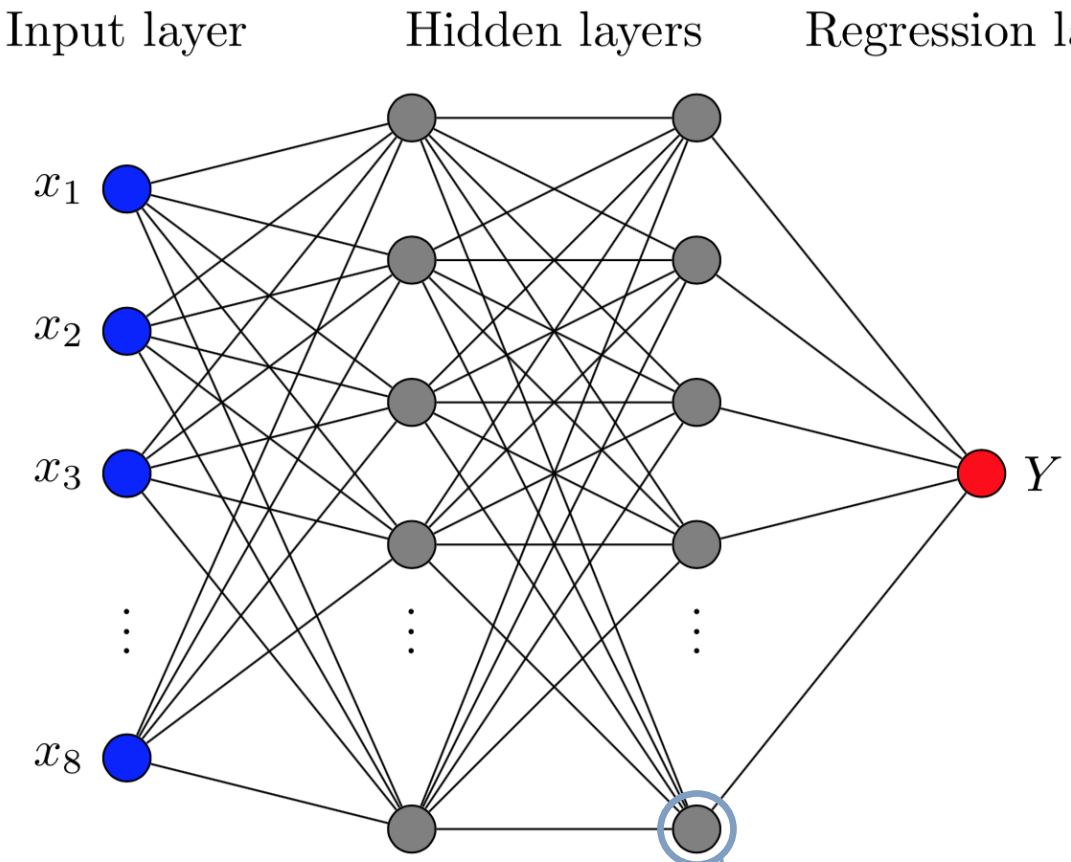


https://ureply.mobi/teacher



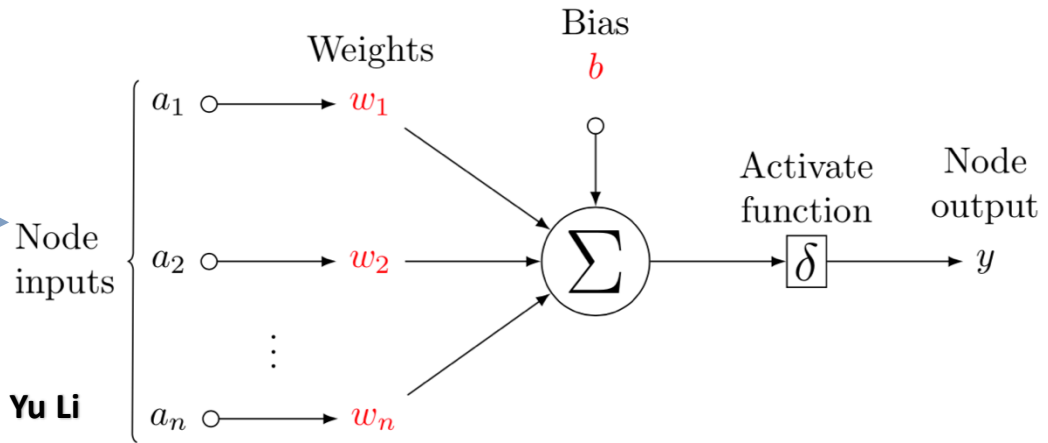
A different angle of looking at Attention

General function approximator



- To resolve complicated problems
  - Increase the number of nodes
  - Increase the number of layers
  - Add non-linear function

- Fully-connected layers
  - A general function approximator
  - We can approximate any function (relation) if we have enough nodes and layers
  - Universal approximation theorem



What is a function?



- ??

- How to represent a function?

What is a function?



- A set X to a set Y assigns to each element of X exactly one element of Y
  - Kind of mapping

- How to represent a function?
  - Mathematical formula
  - Mapping table

How to represent a function?



- Mathematical formula
  - Advantages?
  - Disadvantages?

- Mapping table
  - Advantages?
  - Disadvantages?

How to represent a function?



- Mathematical formula
  - Memory efficient
  - Need to define the form
  - Accuracy of the existing data depends on the form

- Mapping table
  - Precise for the existing data
  - Need to store the data
  - What about the unknown data??

Representing a function without defining the form



- Mapping table
  - Precise for the existing data
  - Need to store the data
  - What about the unknown data??

- How to use mapping table to do prediction?

Person	Height(m)	Weight(kg)	Gender
P1	1.79	75	M
P2	1.64	54	F
P3	1.70	63	M
P4	1.88	78	M
P5	1.75	70	??

Representing a function without defining the form



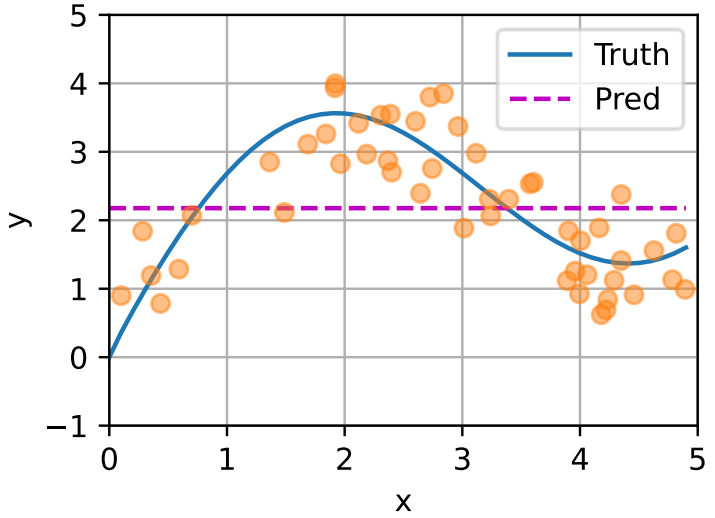
- Mapping table
  - Precise for the existing data
  - Need to store the data
  - What about the unknown data??

- How to use mapping table to do prediction?
  - Using neighbor information
  - More systematic description

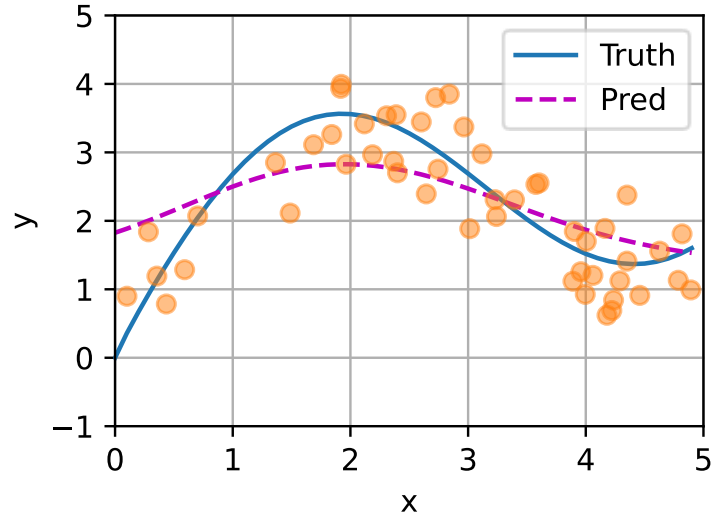
Person	Height(m)	Weight(kg)	Gender
P1	1.79	75	M
P2	1.64	54	F
P3	1.70	63	M
P4	1.88	78	M
P5	1.75	70	??

Different strategies of using neighbor information

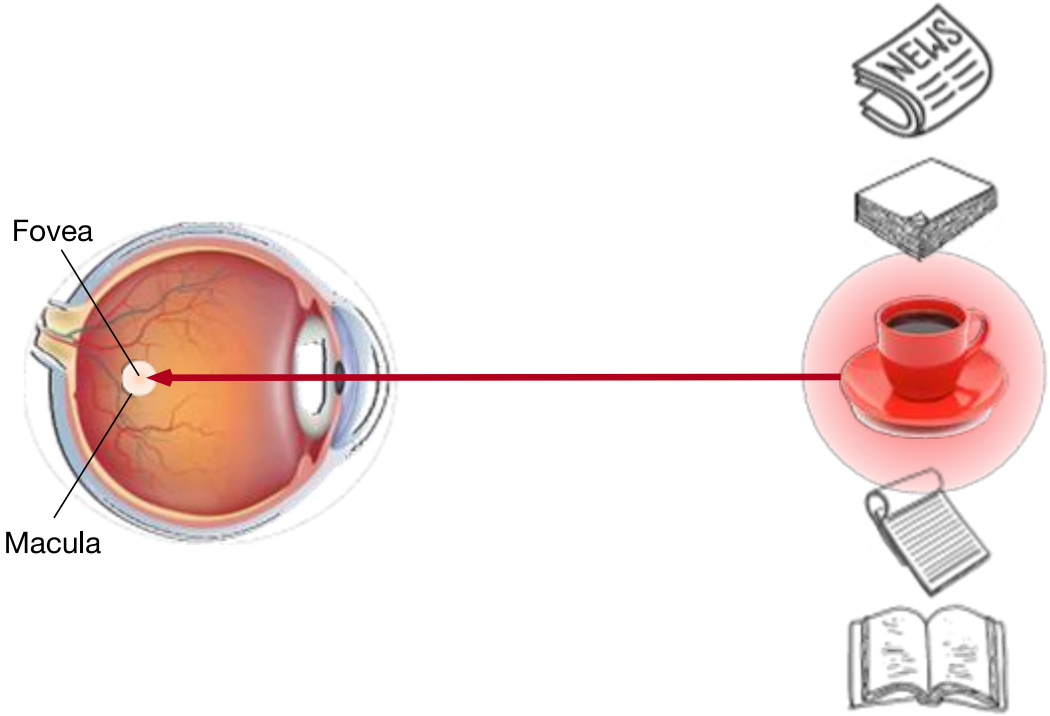
- **Average**
  - $f(x) = \frac{1}{n}\sum_{i=1}^{n} y_i$



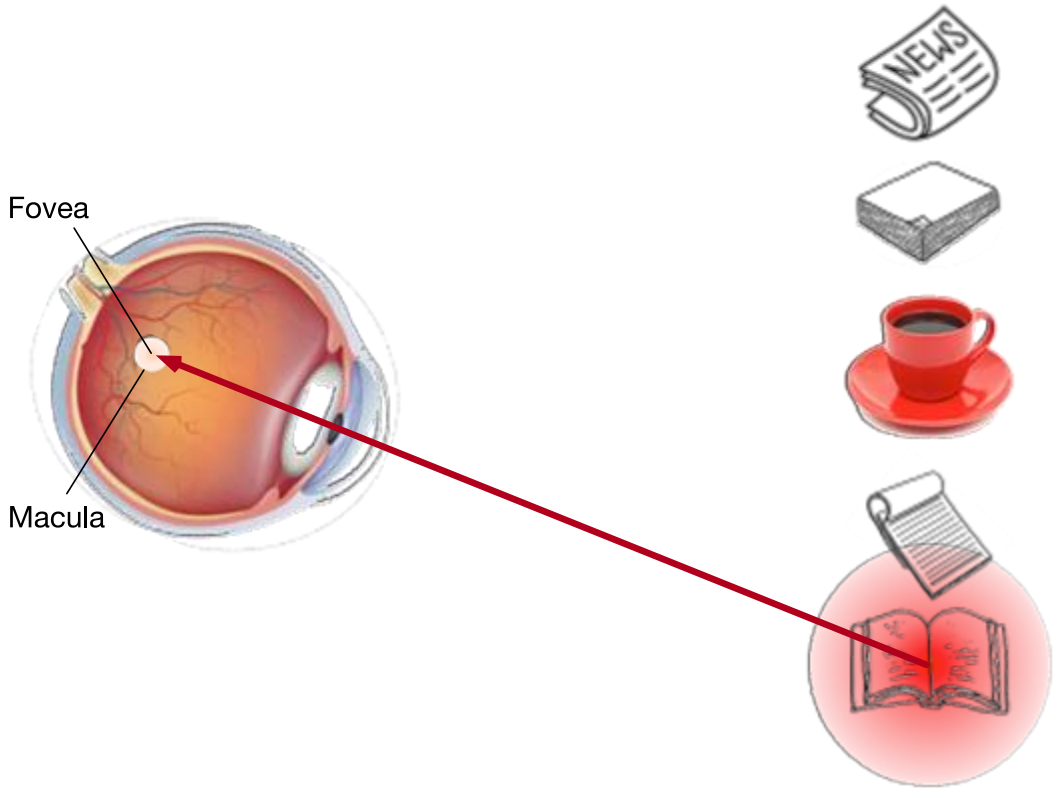
- **Weighted average**
  - $f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$
  - $\alpha(x, x_i)$: related to similarity or distance
  - Even more general: Attention!



Attention

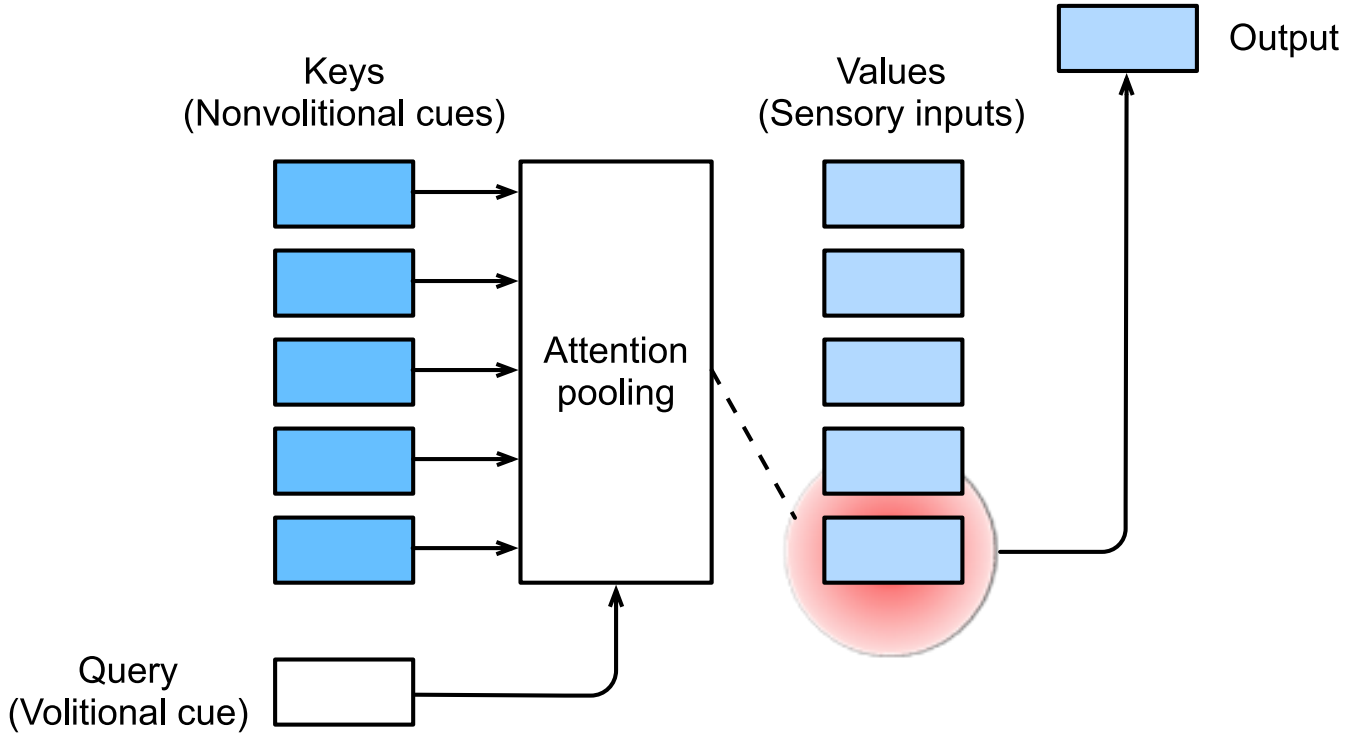


Nonvolitional cue based on saliency



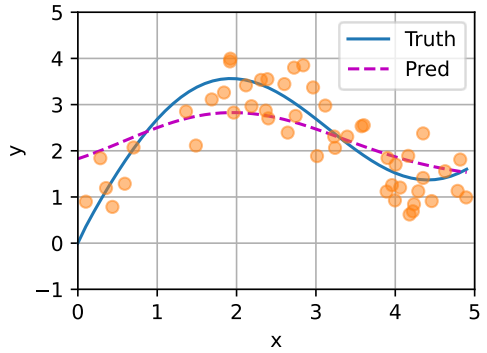
Volitional cue: task dependent

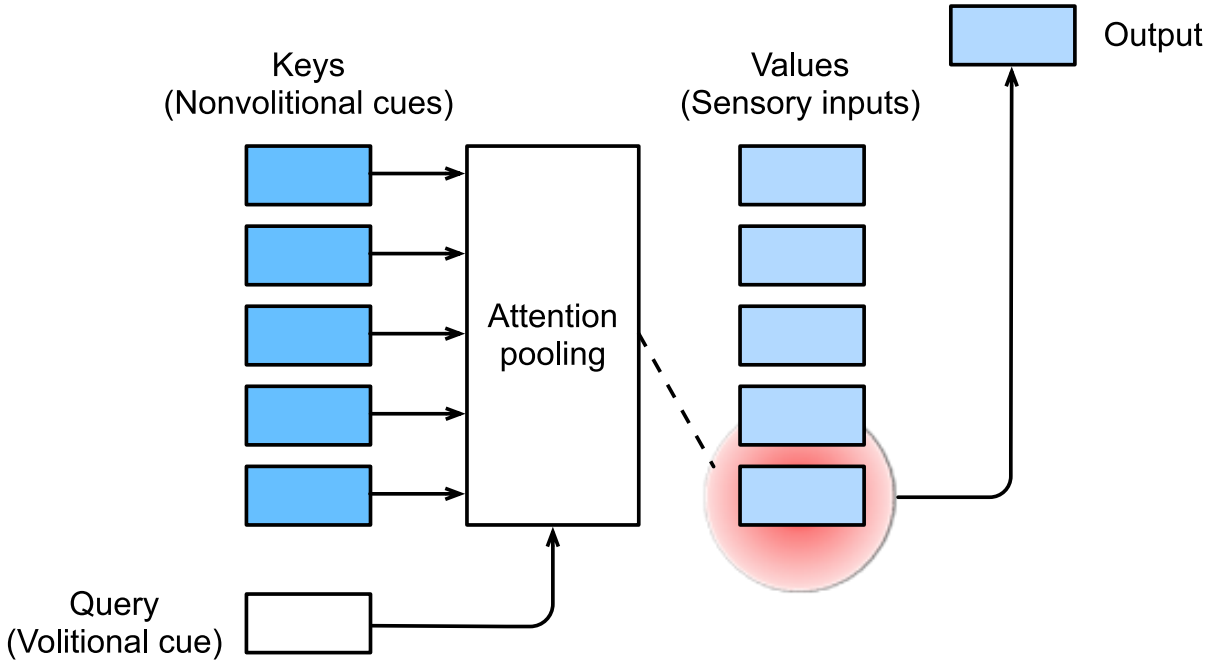
Attention mechanism



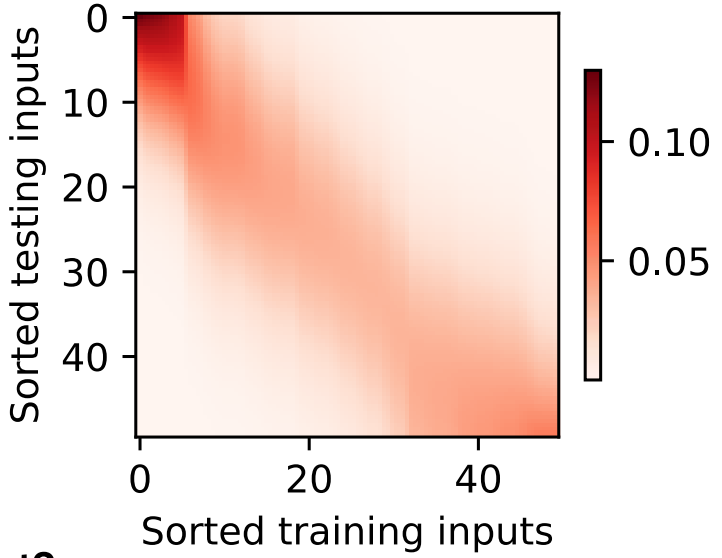
$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

Attention with Gaussian kernel





The weights



Any possible improvement?

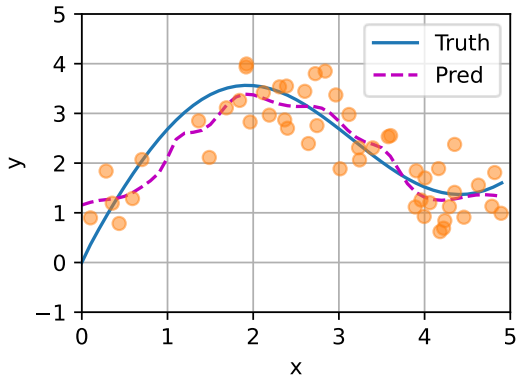
$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

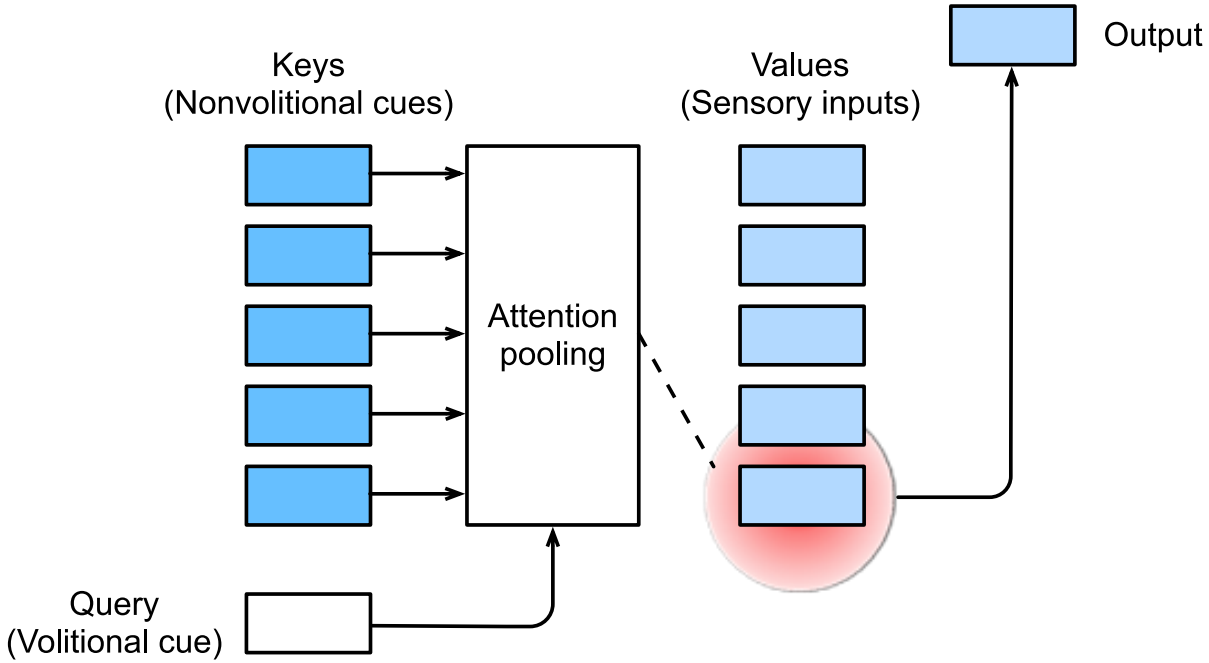
$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{\sum_{j=1}^{n} K(x - x_i)} y_i$$

Let $K(u) = \frac{1}{\sqrt{2\pi}} \exp(-\frac{u^2}{2})$

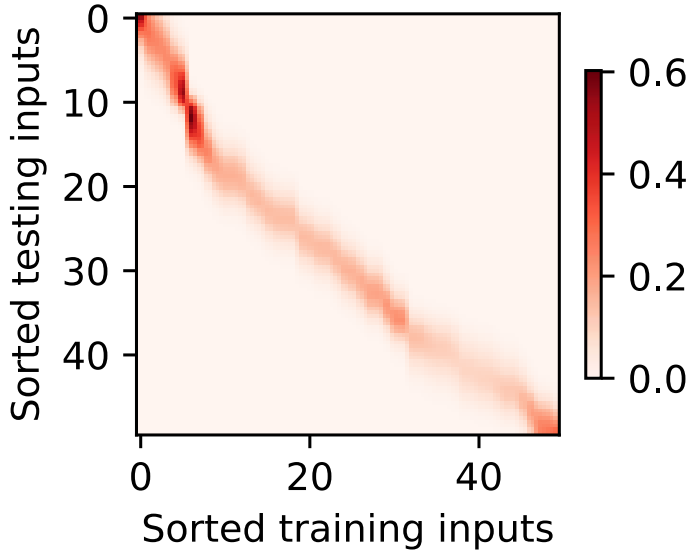
$$f(x) = \sum_{i=1}^{n} \frac{\exp(-\frac{1}{2}(x - x_i)^2)}{\sum_{j=1}^{n} \exp(-\frac{1}{2}(x - x_j)^2)} y_i$$

Attention with parameters





The weights



$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i$$

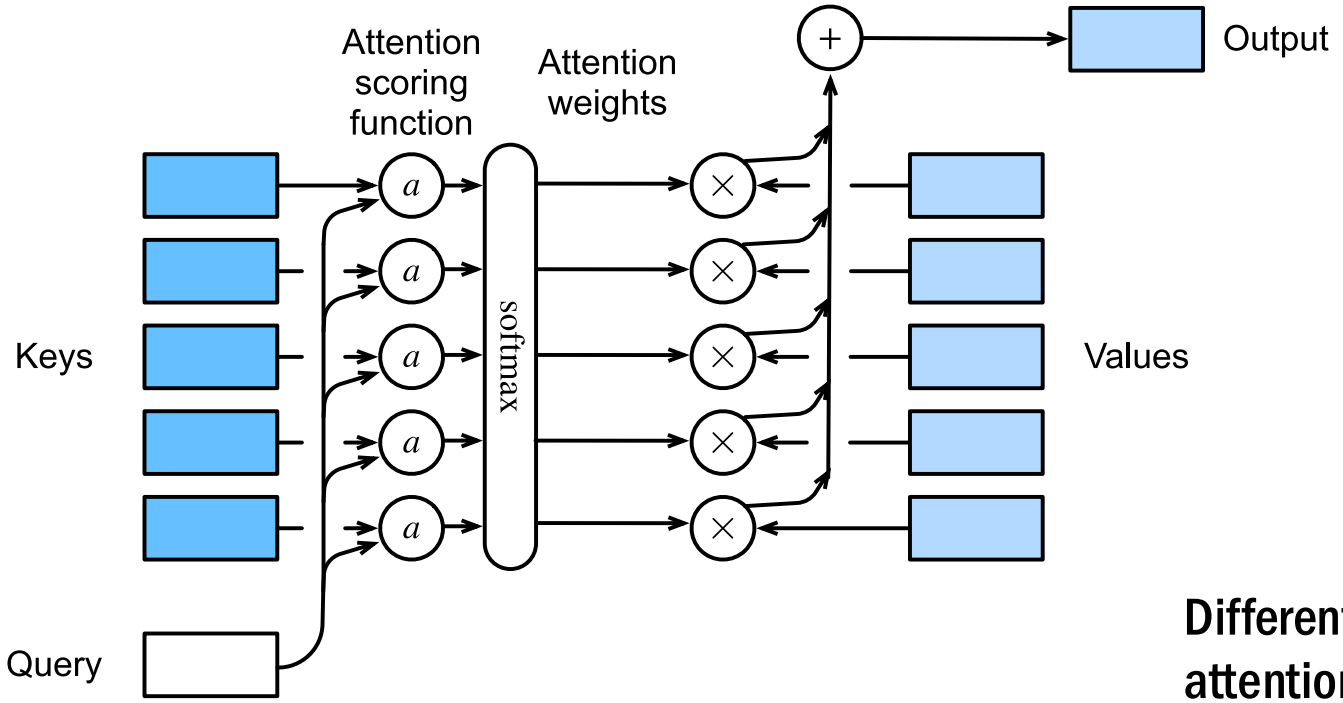
$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{\sum_{j=1}^{n} K(x - x_i)} y_i$$

Let $K(u) = \frac{1}{\sqrt{2\pi}} \exp(-\frac{u^2}{2})$

$$f(x) = \sum_{i=1}^{n} \frac{\exp(-\frac{1}{2}((x - x_i)w)^2)}{\sum_{j=1}^{n} \exp(-\frac{1}{2}((x - x_j)w)^2)} y_i$$

What is this?

The commonly used Attention

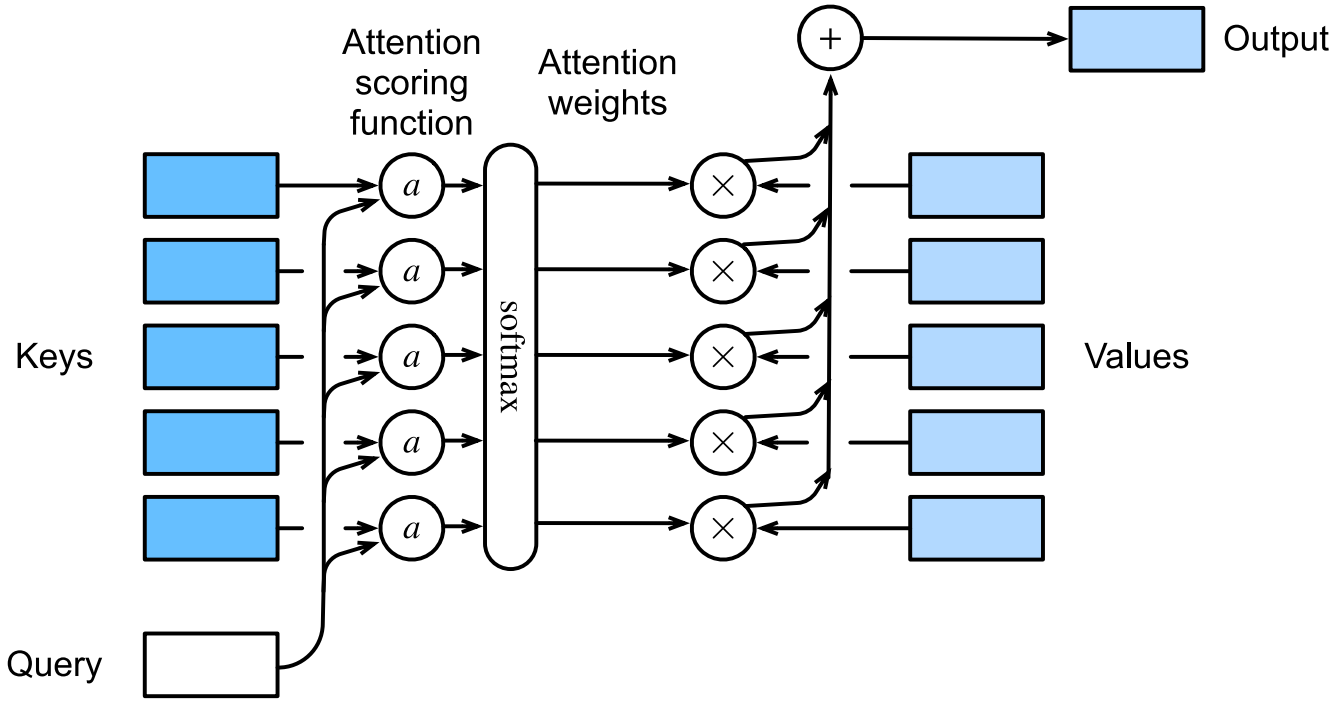


Different a can lead to different attention behaviors

$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i)\boldsymbol{v}_i$$

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = softmax\big(a(\boldsymbol{q}, \boldsymbol{k}_i)\big) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}$$

Additive attention



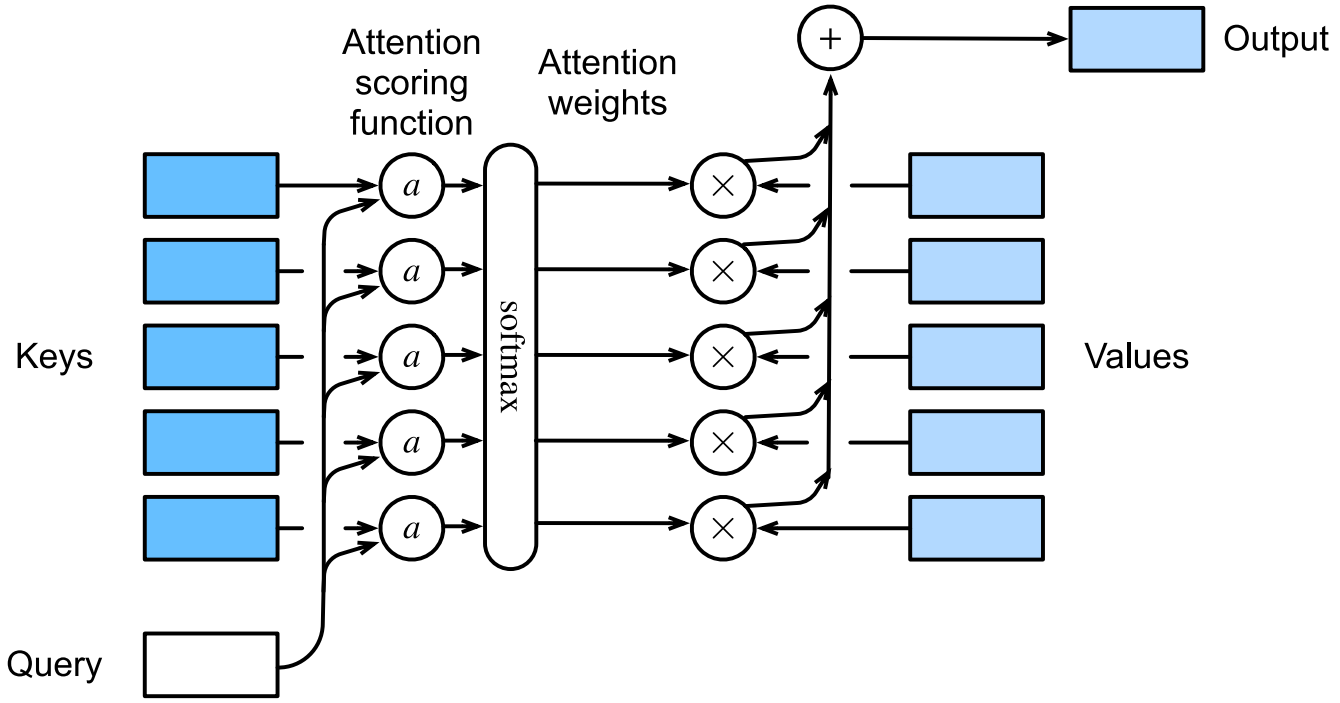
$$a(\boldsymbol{q}, \boldsymbol{k}) = \boldsymbol{w}_v^{\mathrm{T}} \tanh(\boldsymbol{W}_q \boldsymbol{q} + \boldsymbol{W}_k \boldsymbol{k})$$

Usually used when the queries and keys are vectors of different lengths

$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i) \boldsymbol{v}_i$$

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = softmax\big(a(\boldsymbol{q}, \boldsymbol{k}_i)\big) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}$$

Scaled dot-product attention



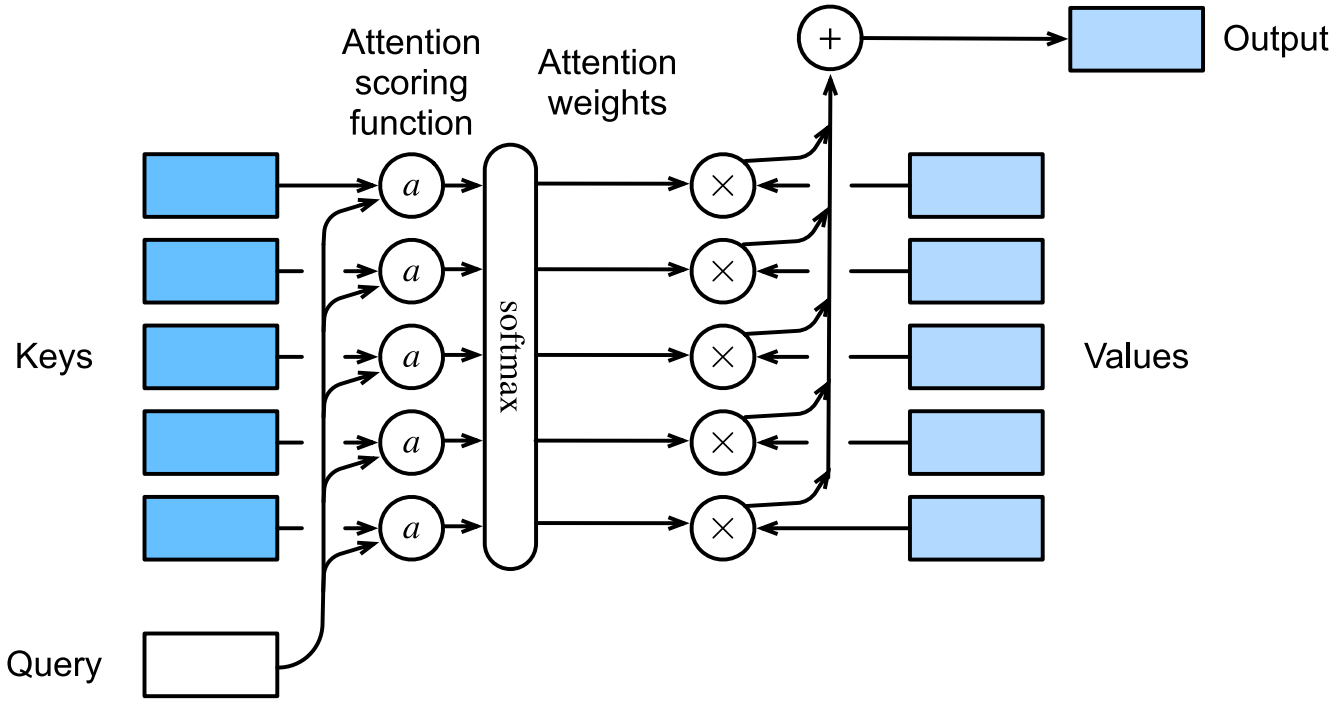
$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i)\boldsymbol{v}_i$$

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = softmax\big(a(\boldsymbol{q}, \boldsymbol{k}_i)\big) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}$$

$$a(\boldsymbol{q}, \boldsymbol{k}) = \frac{\boldsymbol{q}^{\mathrm{T}}\boldsymbol{k}}{\sqrt{\mathrm{d}}}$$

Usually used when the queries and keys are vectors of the same length

The commonly used Attention

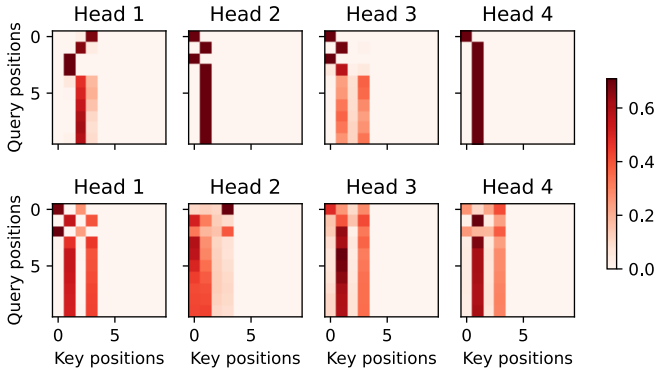


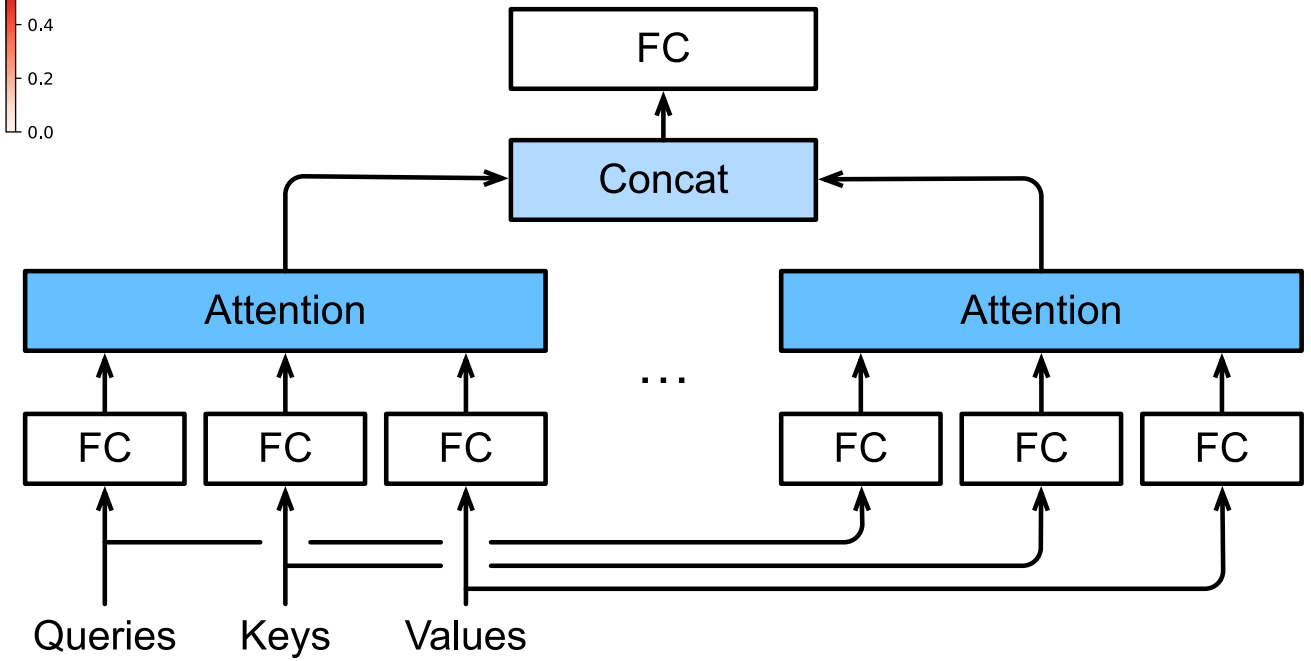
How to improve it?

$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \dots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i)\boldsymbol{v}_i$$

$$\alpha(\boldsymbol{q}, \boldsymbol{k}_i) = softmax\big(a(\boldsymbol{q}, \boldsymbol{k}_i)\big) = \frac{\exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}{\sum_{j=1}^{m} \exp(a(\boldsymbol{q}, \boldsymbol{k}_i))}$$

Multi-head attention

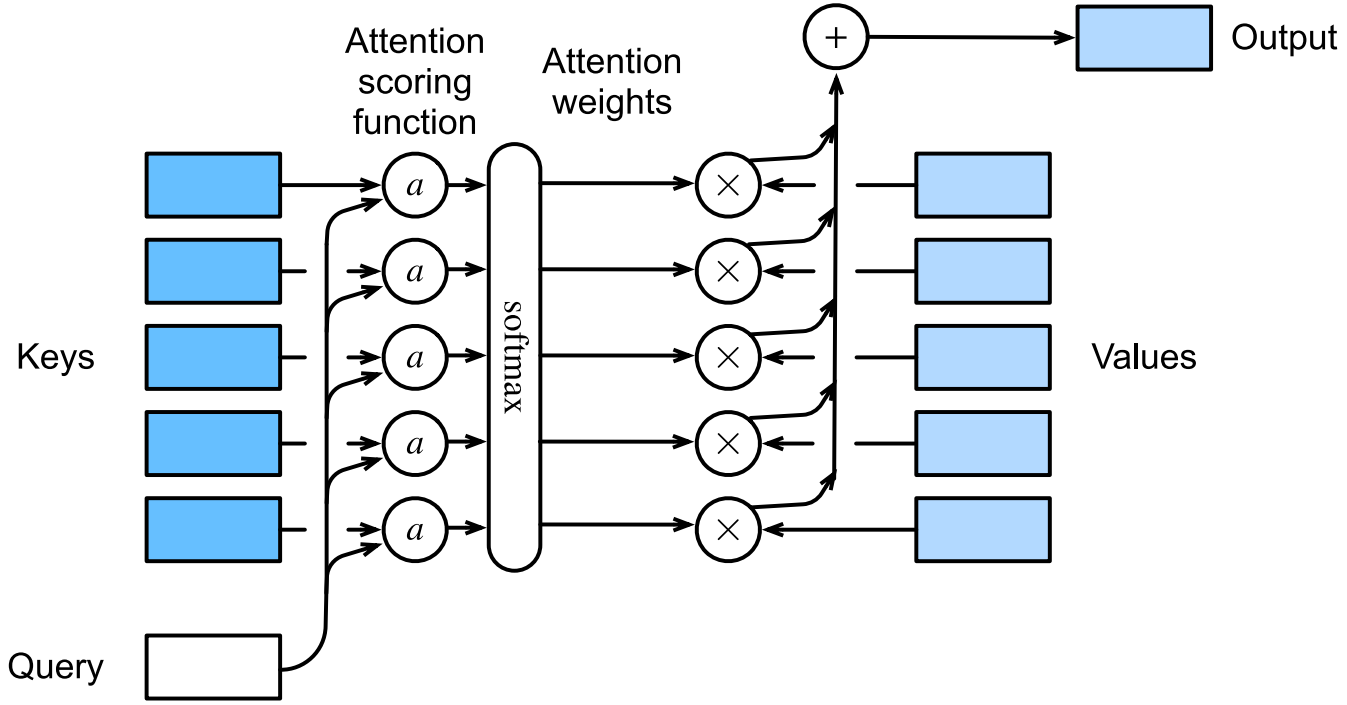




How to deal with sequences?

Different representation subspaces of queries, keys, and values

Self-attention



$$f(\boldsymbol{q}, (\boldsymbol{k}_1, \boldsymbol{v}_1), \ldots, (\boldsymbol{k}_m, \boldsymbol{v}_m)) = \sum_{i=1}^{m} \alpha(\boldsymbol{q}, \boldsymbol{k}_i)\boldsymbol{v}_i$$

Any problem here to deal with sequences?

Embedding of each token: $\boldsymbol{y}_i = f\left(\boldsymbol{x}_i, (\boldsymbol{x}_1, \boldsymbol{x}_1), \ldots, (\boldsymbol{x}_n, \boldsymbol{x}_n)\right)$

$\boldsymbol{x}_i$: input token

Position encoding

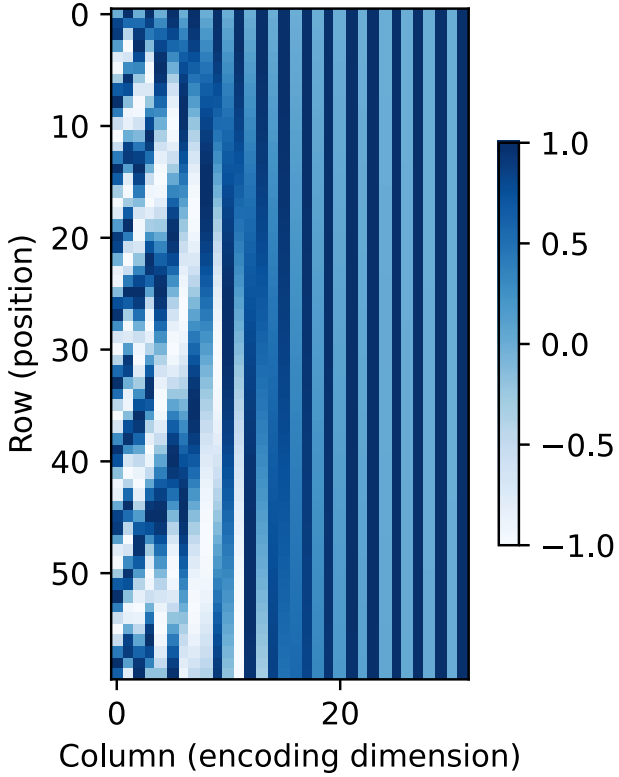


- $\boldsymbol{X} \in R^{n*d}$: d-dimensional embeddings for n tokens of a sequence

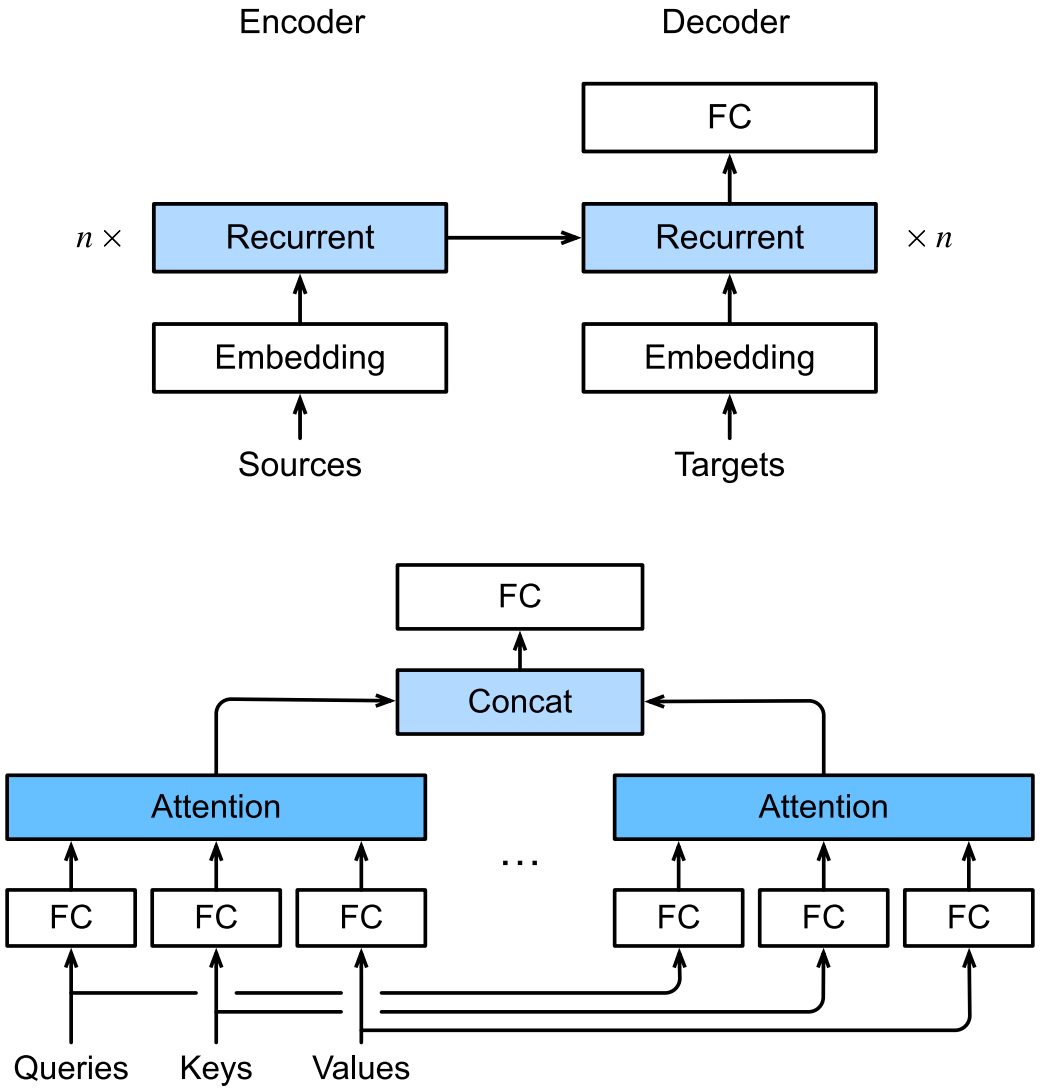
- $\mathbf{P} \in R^{n*d}$
  - The value of the ith row and the $(2j)^{th}$ or the $(2j+1)^{th}$ column
  - $p_{i,2j} = \sin(\frac{i}{10000^{\frac{2j}{d}}})$
  - $p_{i,2j+1} = \cos(\frac{i}{10000^{\frac{2j}{d}}})$

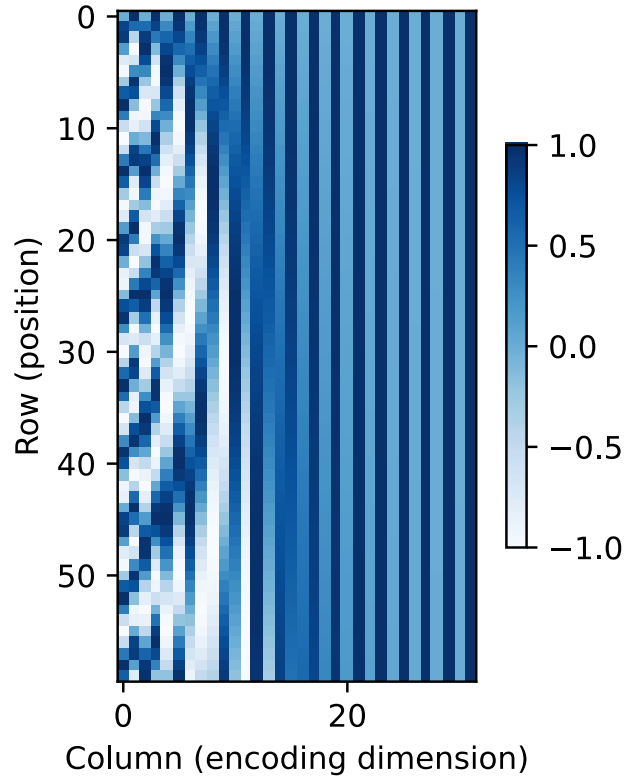
- Weird function but people use it

- Absolute & relative positional information

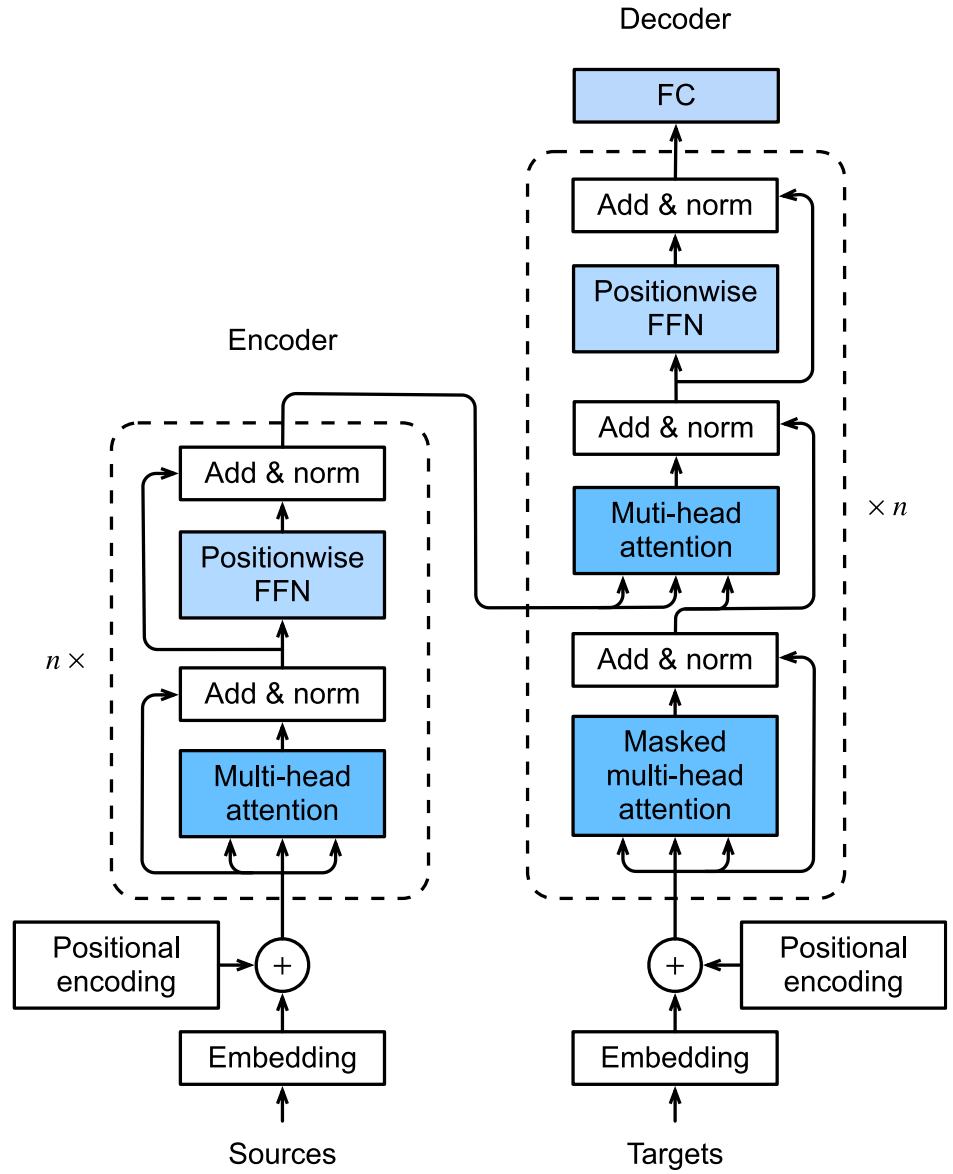


Put everything together





Transformer



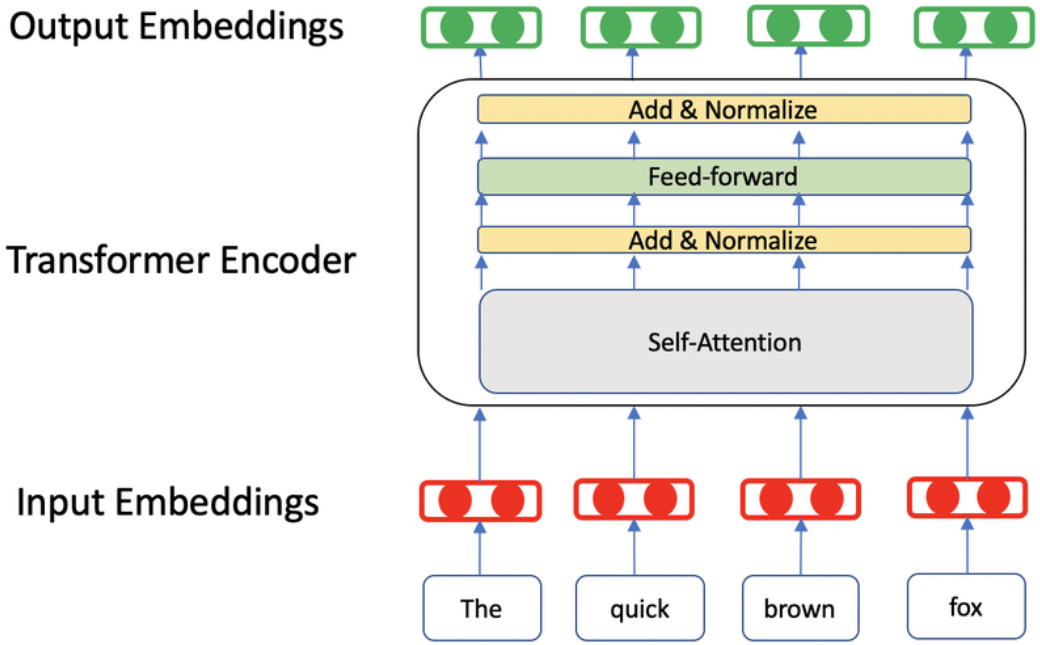
Masked attention: do not consider padding position

Dimension at each layer?

Functions of Transformer



- After learning the hidden state (embedding)
  - Machine translation
  - Token-level prediction
  - Sentence-level prediction
  - Text generation
  - …
- If we have the labelled data
  - Supervised learning
  - Cross-entropy loss
- But what if we only have corpus? No label?
  - Loss function?



Resource and advanced topics



- Dive into deep learning: Chapter 11

- Framewise phoneme classification with bidirectional LSTM and other neural network architectures
- Sequence to sequence learning with neural networks
- Learning phrase representations using RNN encoder-decoder for statistical machine translation
- Attention is All you Need

Post-lecture survey



- https://forms.gle/JCgMasYuTrAjK4VA9

Next time

- BERT





Thank you!

Yu LI (李煜)

liyu95.com

liyu@cse.cuhk.edu.hk

The Artificial Intelligence in Healthcare (AIH) Group

Department of Computer Science and Engineering (CSE)

The Chinese University of Hong Kong (CUHK)